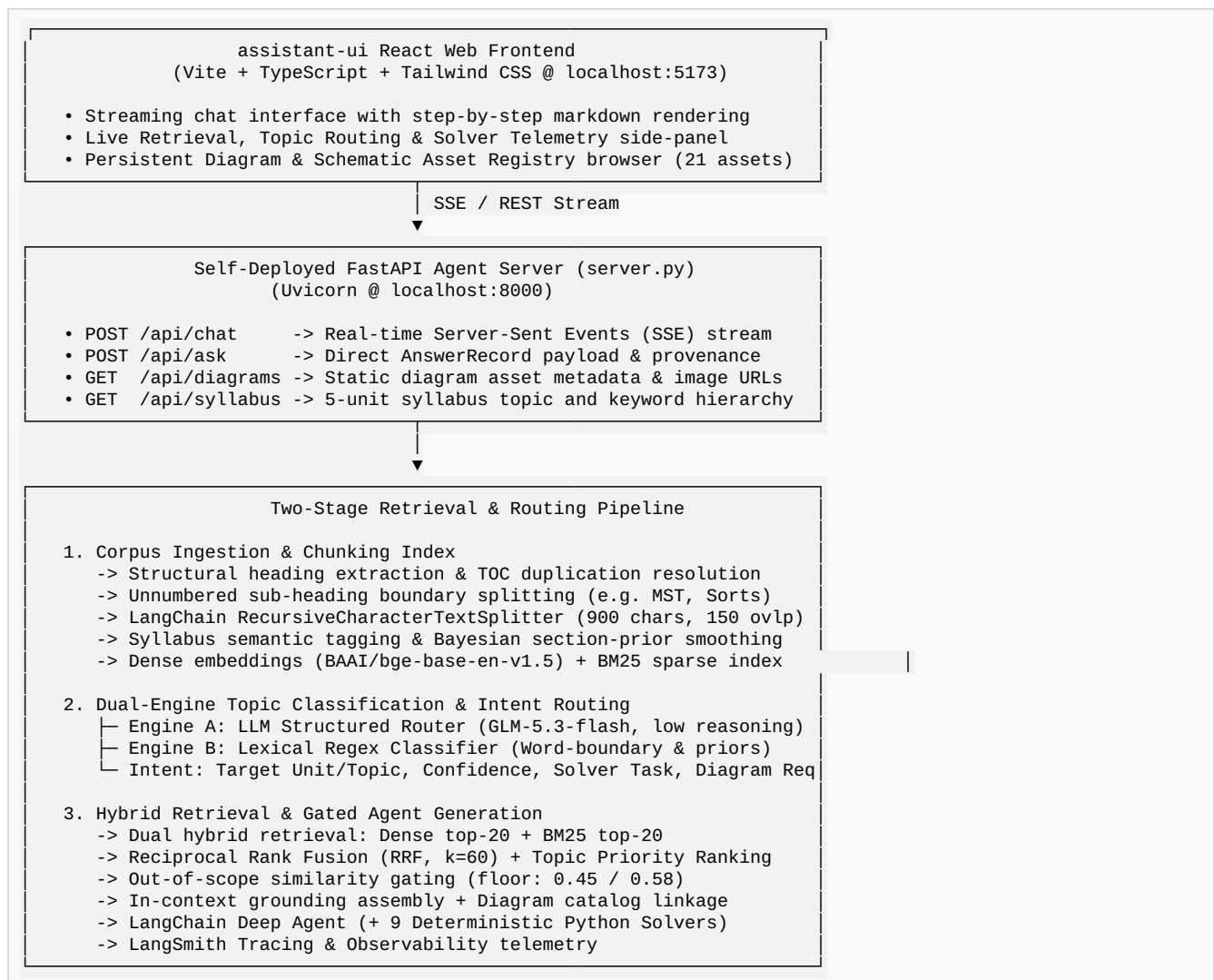


ENGINEERING CALIBRATION AUDIT - RAG BASED AI APPLICATION

I. SYSTEM ARCHITECTURE OVERVIEW

The system is a two-stage retrieval augmented generation pipeline coupled with a self-deployed FastAPI Agent Server, LangChain/LangSmith telemetry tracing, and an assistant-ui web interface. Retrieval is fully deterministic and completes prior to generation, ensuring that all passages, similarity metrics, and solver traces recorded in this report represent the exact grounding context received by the language model. Generation is executed by a LangChain Deep Agent whose mathematical operations are strictly delegated to exact deterministic Python solvers.

A. End-to-End System Architecture



B. Corpus Ingestion, Heading Extraction, and Chunk Construction Pipeline

To achieve complete syllabus fidelity without contextual drift, corpus construction follows a strict multi-tier parsing and windowing pipeline:

- Heterogeneous Source Ingestion:** - **Supplied Core Notes:** 1038678511-Daa-Aktu-Notes.txt (44 chunks). - **Authored Supplementary Notes:** notes_supplement.md (22 chunks), systematically authored to close curricular coverage gaps (LCS, Floyd-Warshall, Backtracking, Branch & Bound, Tries, Skip Lists, Linear-time sorting, Randomized algorithms, FFT, Job Sequencing). - **Official Syllabus Document:** syllabus-daa.pdf (4 chunks) providing formal topic definitions and learning outcomes. - **Previous Year Question Papers:** pyq/*.pdf (37 chunks across multiple examination years). - **Strict Isolation Policy:** A total of 107 chunks are indexed. Exactly 70 chunks form the answerable

corpus. PYQ chunks are indexed strictly for authentic test-case extraction and provenance auditing, but are strictly quarantined (`source_type != "pyq"`) from retrieval pools to prevent question-question context pollution.

2. **Heading Extraction & Sub-Heading Boundary Resolution:** - The primary study notes use formatted section headings (e.g., 1.2 Recurrences & Master Theorem). A regex parser (`SECTION_RE`) extracts numbered section headers while resolving Table of Contents duplication by preserving only the trailing, real-body section instance. - Crucially, unnumbered sub-topics (such as Minimum Spanning Tree (MST) or Merge Sort) that appear as bare title lines within larger sections are detected via `SUBHEAD_RE ( r"^(?!\\s*[-*]) [ \\t]*([A-Z][A-Za-z0-9 '()\\-./&.,]{2,55}) [ \\t]*$\\n\\s*\\n"`). This splits the section body and re-attributes sub-headings (e.g., 3.1 Greedy Algorithms > Minimum Spanning Tree (MST)), preventing algorithmic sub-topics from inheriting misleading parent headers.
3. **LangChain Text Windowing & Boundary Parameterization:** - Sections are partitioned using LangChain's `RecursiveCharacterTextSplitter` configured with a target `chunk_size = 900` characters and `chunk_overlap = 150` characters. - Separators follow the structural precedence [`"\\n\\n", "\\n", ". ", " ", ""`], ensuring that natural paragraph breaks and algorithm step boundaries are preserved intact. - Micro-fragments under 40 characters are automatically purged to prevent index dilution.
4. **Syllabus Keyword Priors & Bayesian Section-Level Smoothing:** - Every chunk is scored against the AKTU KCS-503 syllabus taxonomy using lookaround word-boundary regexes (`r"(<?!\\w)" + re.escape(kw) + r"s?(?!\\w)"`). Matches against section headings receive a $3\times$ weight multiplier, while "strong" disambiguation keywords (e.g., relaxation vs. negative cycle) receive a $3\times$ body weight and a $9\times$ heading weight. - **Section Prior Smoothing:** When a chunk's local keyword score is weak or transiently mentions an adjacent topic (e.g., a Heap section referencing Prim's MST in passing), a Bayesian section-prior margin (`SECTION_PRIOR_MARGIN = 12.0`) guarantees that the topic computed over the entire parent section prevails, preventing localized chunk misclassification. - Each chunk is permanently tagged with its unique ID (`[C0001]`), unit number, topic name, source document, section heading, and computed topical confidence score.

C. Dual-Engine Topic Classification and Disambiguation Architecture

To prevent cross-unit terminology collisions (such as "relaxation" appearing in both Unit 3 Dijkstra and Unit 4 Bellman-Ford, or "knapsack" spanning Unit 3 Greedy and Unit 4 Dynamic Programming), the system employs a synchronized dual-engine topic classifier:

1. **Deterministic Lexical Classifier (`lexical_classify`):** - Matches query tokens against the 29 syllabus topic dictionaries using word-boundary regular expressions. - Normal keywords contribute 1 point; "strong" disambiguation anchors (e.g., "negative weight" $\rightarrow$ Bellman-Ford) contribute 4 points. - Computes a dynamic confidence score based on the margin over the second-best topic and total keyword evidence:
$$\text{margin} = \frac{\text{score}\{\text{top}\} - \text{score}\{\text{runner}\}}{\text{score}\{\text{top}\}}, \quad \text{evidence} = \min\left(1.0, \frac{\text{score}\{\text{top}\}}{4.0}\right)$$

$$\text{Confidence}_{\text{lexical}} = \min\left(0.95, (0.45 + 0.5 \times \text{margin}) \times \text{evidence}\right)$$
 - Queries with balanced keyword collisions or isolated weak keywords receive low confidence, preventing false triggers.
2. **LLM Structured Classifier (`llm_classify`):** - Dispatches a prompt containing the 29-topic syllabus catalog to GLM-5.3-flash with `reasoning_effort="low"` to strictly return a schema-validated JSON payload: `{"topic": "<topic_id or null>", "confidence": <0..1>, "needs_diagram": <bool>, "numeric_task": "<solver_name or null>"}` - Leverages full semantic reasoning to resolve ambiguous student formulations where keywords overlap multiple units.
3. **Classification Agreement and Fusion (`classify`):** - When both engines agree on the target topic, confidence is boosted:
$$\text{Confidence} = \min(0.99, \max(\text{Confidence}\{\text{llm}\}, \text{Confidence}\{\text{lexical}\}), 0.8)$$
 with telemetry source tagged as "llm+lexical". - When the LLM resolves a valid syllabus topic where lexical keywords are ambiguous, the LLM classification takes precedence with source "llm". - When neither engine identifies a syllabus match, confidence is set to 0.0\$, routing the query toward the out-of-scope similarity gate.
4. **Intent & Tool Task Routing:** - **Numerical Task Detection:** Scans for computation intent markers (`COMPUTE_INTENT` : "solve", "calculate", "trace", "step by step") combined with numeric data or specific solver hints (`NUMERIC_HINTS`) to automatically bind one of the 9 deterministic Python solvers (e.g., `knapsack_01`, `floyd_warshall`, `master_theorem`). - **Diagram Intent Detection:** Scans for visual phrasing (`DIAGRAM_HINTS` : "diagram", "draw", "figure", "tree", "stages") or model intent to inject eligible schematic identifiers.

D. GLM-5.3-Flash Chunk Access, Hybrid Retrieval, and End-to-End Query Flow

When a student submits a query, the system orchestrates chunk retrieval and model grounding through an 8-stage deterministic execution flow:

1. **Query Ingestion & Dual Topic Routing:** The query is processed by the dual-engine classifier detailed above, determining target Unit, Topic, confidence, and solver bindings.
2. **Dual-Stream Hybrid Retrieval (Dense + Sparse):** - **Dense Semantic Stream:** Query is prepended with "Represent this sentence for searching relevant passages: " and embedded with BAAI/bge-base-en-v1.5 into a 768-dimensional normalized vector. Cosine similarity is computed against all 70 answerable chunk vectors. - **Sparse Lexical Stream:** Query tokens are evaluated against the inverted BM25Okapi index over chunk headings and body text. - Top-20 candidates are retrieved independently from each stream.
3. **Reciprocal Rank Fusion (RRF):** Rankings are merged into a unified candidate pool using: $RRF(d) = \sum_{m \in [dense, bm25]} \frac{1}{(60 + r_m(d))}$ where $r_m(d)$ represents the 1-indexed rank of chunk d in retrieval stream m .
4. **Topic Priority Re-ranking & Adaptive Context Sizing:** When topic confidence ≥ 0.7 , matching syllabus topic chunks receive a $1.5\times$ priority boost to lead the context window. Adaptive backfilling retrieves general passages only as needed to maintain a 3-chunk minimum context depth.
5. **Out-of-Scope Similarity Floor Gating:** Cosine similarity floors (\$0.45\$ for topic-classified queries, \$0.58\$ for unclassified queries) deterministically intercept off-syllabus queries and issue formal syllabus refusals without consuming generation tokens.
6. **Dynamic In-Context Grounding Formulation:** Top-\$k\$ chunks are formatted into an explicit grounding block with chunk IDs ([c0042]), metadata, eligible diagram IDs, and numerical solver directives.
7. **LangChain Deep Agent Execution & Solver Delegation:** GLM-5.3-flash operates under zero-arithmetic guardrails, invoking deterministic Python solvers for exact step-by-step tables and recurrence evaluations.
8. **Post-Generation Diagram Verification & Streaming:** A post-processing regex pass verifies and strips unoffered diagram references before streaming response markdown to assistant-ui over SSE.

E. Context Management, Thread State, and LangSmith Observability

1. **Per-Turn Grounding Window:** The model receives only the verified grounding context for the active query, preventing multi-turn context drift and token window degradation.
2. **Thread State Persistence:** The server records complete turn telemetry: classified topic, routing source, full chunk payloads with dense/BM25/RRF scores, executed tool names and argument payloads, emitted diagram IDs, and execution latency.
3. **LangChain / LangSmith Tracing Telemetry:** The pipeline natively integrates LangChain Tracing V2 (LANGCHAIN_TRACING_V2=true). Every retrieval step, LLM completion, tool invocation, and agent trajectory is automatically traced and logged to LangSmith under the rag-aktu project for granular latency profiling, token accounting, and live auditability.

F. Syllabus Coverage and Authored Supplement

The knowledge base covers all 29 syllabus topics across the 5 units of AKTU KCS-503. Topics omitted from the primary notes (LCS detail, Floyd-Warshall, backtracking, branch & bound, tries, skip lists, linear time sorting, randomized algorithms, FFT, job sequencing) are authored in notes_supplement.md and explicitly cited.

G. Generation Model and Environment Configuration

- **Generation Model:** z-ai/glm-5.3-flash via openai_compat.
- **Embedding Model:** BAAI/bge-base-en-v1.5 (768 dimensions, normalized).
- **Hybrid Fusion:** Dense (Cosine) + Sparse (BM25Okapi), fused via RRF (\$k=60\$).
- **Classifier Engines:** GLM-5.3-flash structured router + Regex lexical scorer.
- **Tool Suite:** 9 Deterministic Python Solvers (rag.solvers).
- **Observability:** LangSmith Tracing V2 + FastAPI SSE Streaming.

II. DIAGRAM AND ASSET ID INDEX

Every visual schematic in the system carries a persistent unique identifier following the specification DIAG_DAA_U{unit}_{CONCEPT}_{NN}. The identifier is deterministic and defined in data/diagrams/diagrams.json (21 assets total).

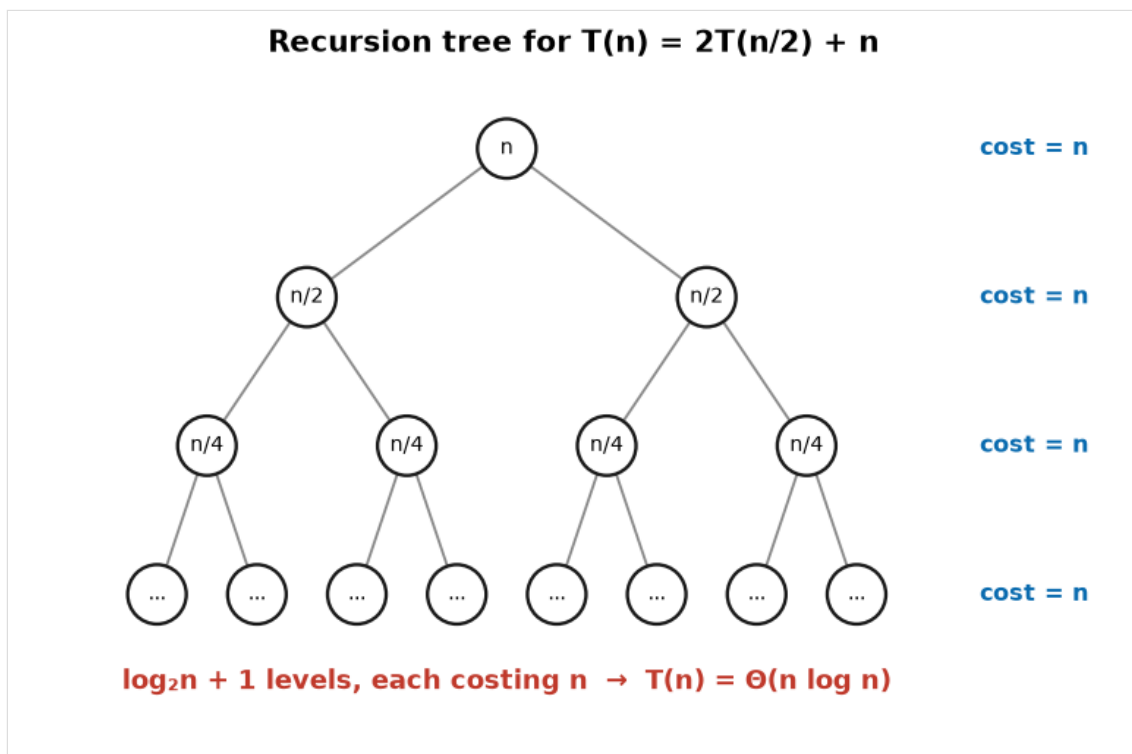
Assets depicting algorithmic traces or dynamic programming tables (knapsack, LCS, Floyd-Warshall, matrix chain, Dijkstra, Bellman-Ford) are programmatically rendered by executing the exact solver code invoked by the agent, ensuring complete mathematical concordance between text and schematics.

A. Asset Registry Catalog

DIAGRAM ID	UNIT	TOPIC	CONCEPT	FILENAME
DIAG_DAA_U1_RECUR_TREE_01	Unit 1	recurrences	recursion tree for $T(n)=2T(n/2)+n$	u1_recur_tree_01.png
DIAG_DAA_U1_MASTER_CASES_01	Unit 1	recurrences	Master Theorem three cases flowchart	u1_master_cases_01.png
DIAG_DAA_U1_MERGE_SORT_01	Unit 1	sorting_comparison	merge sort divide and merge stages	u1_merge_sort_01.png
DIAG_DAA_U2_AVL_ROT_01	Unit 2	red_black_trees	AVL LL single right rotation	u2_avl_rot_01.png
DIAG_DAA_U2_AVL_ROT_02	Unit 2	red_black_trees	AVL LR double rotation	u2_avl_rot_02.png
DIAG_DAA_U2_RB_INSERT_01	Unit 2	red_black_trees	red-black tree insertion rebalancing	u2_rb_insert_01.png
DIAG_DAA_U2_BTREE_SPLIT_01	Unit 2	b_trees	B-tree node split on the median key	u2_btree_split_01.png
DIAG_DAA_U2_BINOMIAL_HEAP_01	Unit 2	binomial_fibonacci_heaps	binomial trees B0 B1 B2 structure	u2_binomial_heap_01.png
DIAG_DAA_U3_BFS_DFS_01	Unit 3	graph_traversal	BFS and DFS visit order	u3_bfs_dfs_01.png
DIAG_DAA_U3_MST_KRUSKAL_01	Unit 3	mst	Kruskal MST construction stages	u3_mst_kruskal_01.png
DIAG_DAA_U3_MST_PRIM_01	Unit 3	mst	Prim MST growth stages	u3_mst_prim_01.png
DIAG_DAA_U3_DIJKSTRA_01	Unit 3	dijkstra	Dijkstra distance array trace	u3_dijkstra_01.png
DIAG_DAA_U3_BELLMAN_01	Unit 3	bellman_ford	Bellman-Ford pass by pass distances	u3_bellman_01.png
DIAG_DAA_U3_HUFFMAN_01	Unit 3	huffman_coding	Huffman coding tree with codewords	u3_huffman_01.png
DIAG_DAA_U4_KNAPSACK_TABLE_01	Unit 4	knapsack_01	0/1 knapsack dynamic programming table	u4_knapsack_table_01.png
DIAG_DAA_U4_LCS_TABLE_01	Unit 4	lcs	LCS length table with direction arrows	u4_lcs_table_01.png
DIAG_DAA_U4_FLOYD_MATRIX_01	Unit 4	floyd_warshall	Floyd-Warshall distance matrices	u4_floyd_matrix_01.png
DIAG_DAA_U4_MCM_TABLE_01	Unit 4	matrix_chain	matrix chain multiplication cost table	u4_mcm_table_01.png
DIAG_DAA_U4_NQUEEN_TREE_01	Unit 4	backtracking	4-queens state space tree	u4_nqueen_tree_01.png
DIAG_DAA_U5_KMP_PREFIX_01	Unit 5	string_matching	KMP prefix function table	u5_kmp_prefix_01.png
DIAG_DAA_U5_NP_CLASSES_01	Unit 5	np_completeness	P NP NP-complete NP-hard relationship	u5_np_classes_01.png

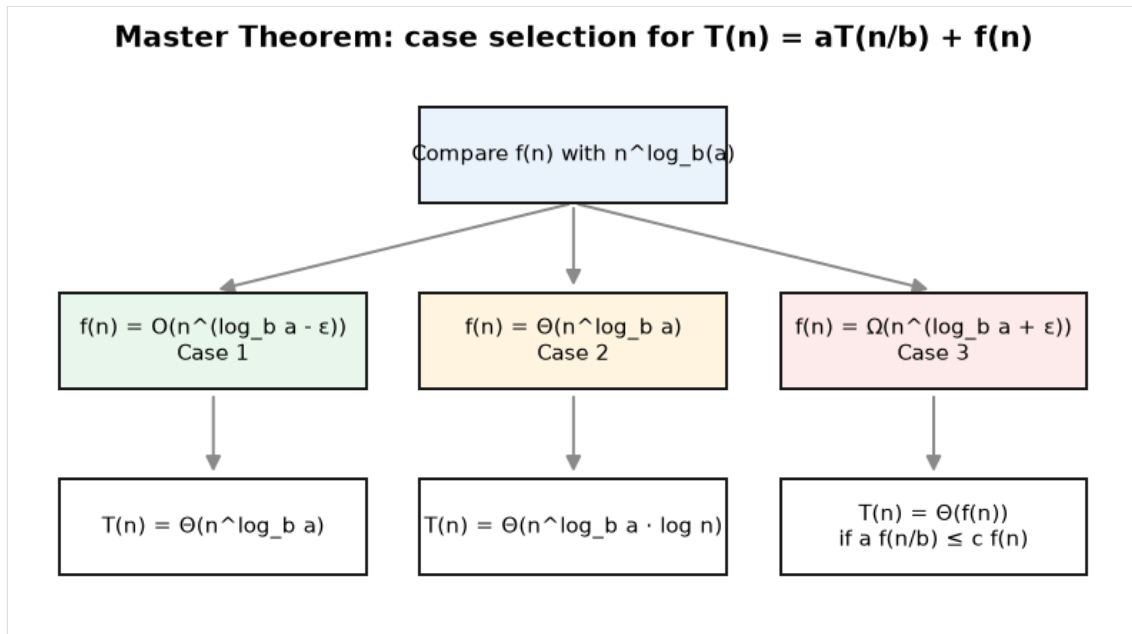
B. Rendered Diagram Schematics

Fig. 1: DIAG_DAA_U1_RECUR_TREE_01 — recursion tree for $T(n)=2T(n/2)+n$



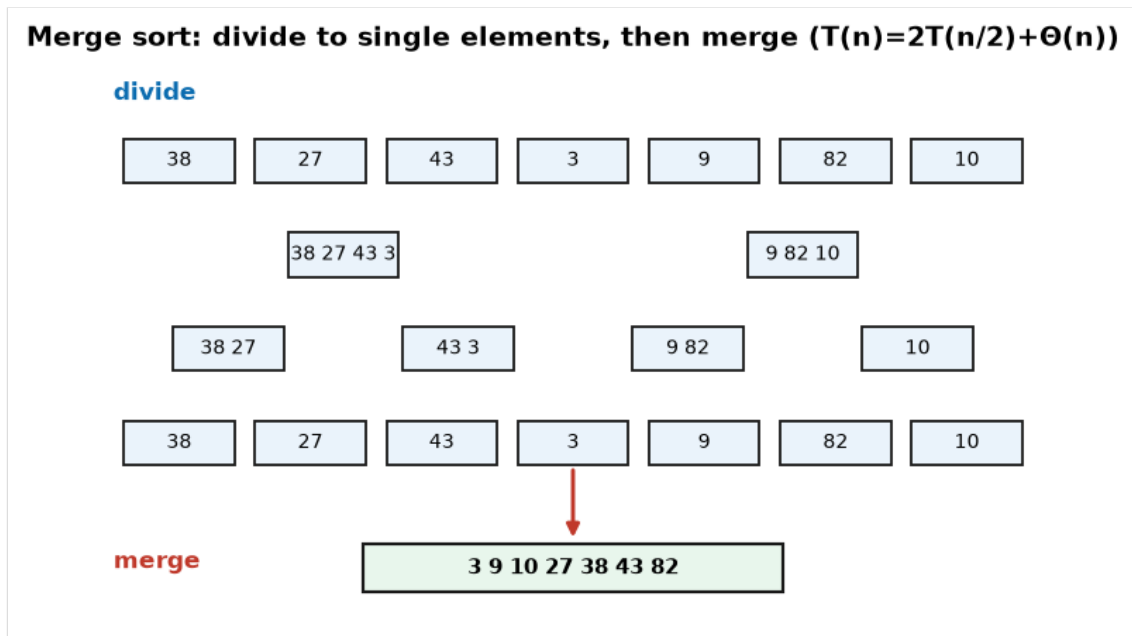
Recursion tree expansion showing $\log n$ levels of cost n each, giving $\Theta(n \log n)$.

Fig. 2: DIAG_DAA_U1_MASTER_CASES_01 — Master Theorem three cases flowchart



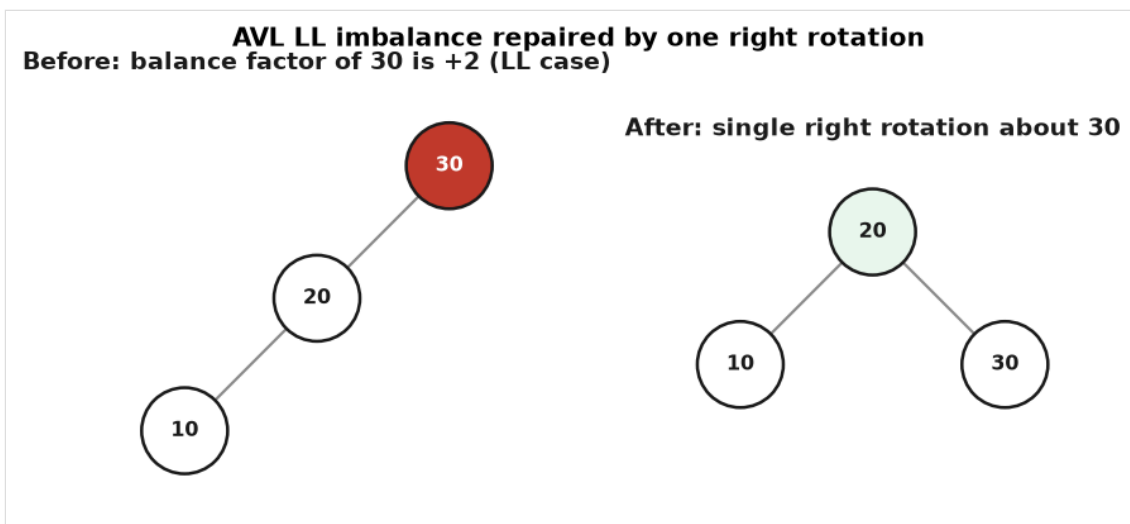
Decision chart for the three cases of the Master Theorem and the bound each yields.

Fig. 3: DIAG_DAA_U1_MERGE_SORT_01 — merge sort divide and merge stages



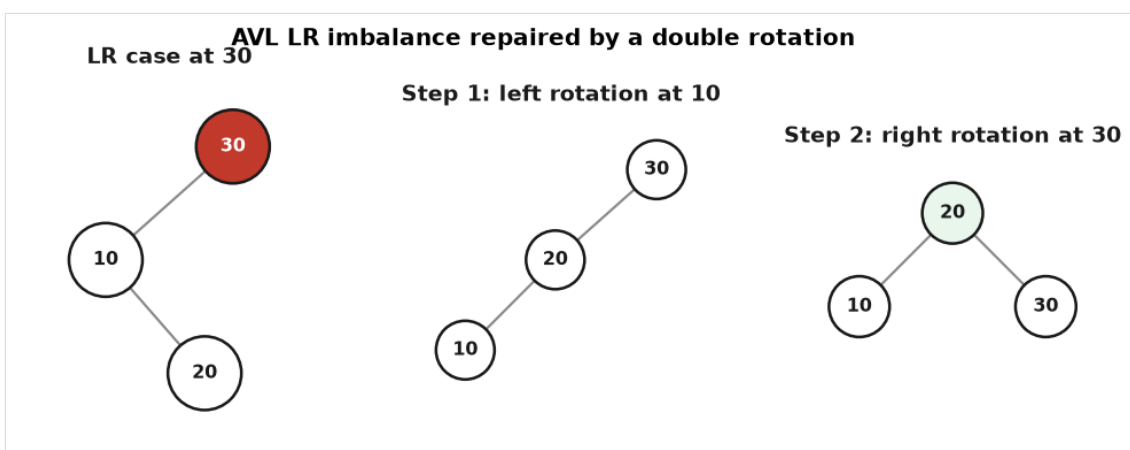
Merge sort splitting an array to singletons and merging back in sorted order.

Fig. 4: DIAG_DAA_U2_AVL_ROT_01 — AVL LL single right rotation



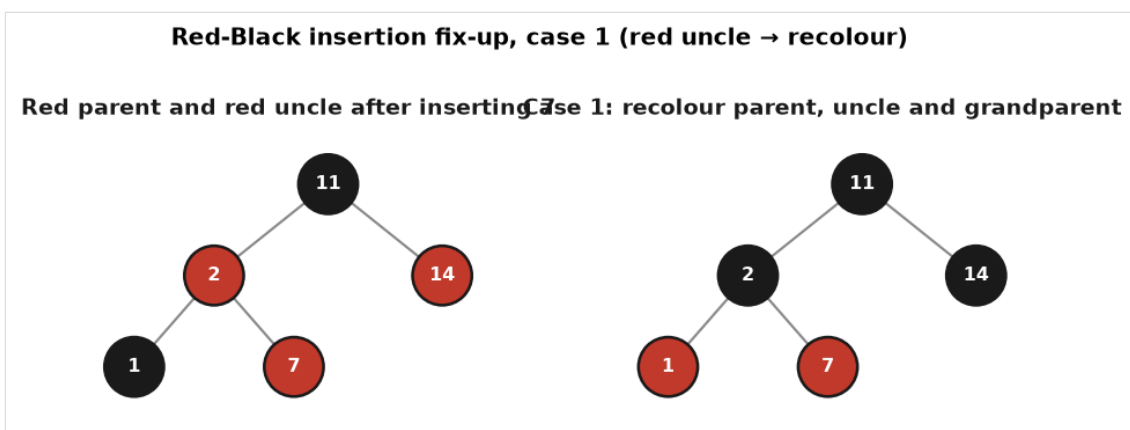
An LL imbalance and the single right rotation that restores the AVL property.

Fig. 5: DIAG_DAA_U2_AVL_ROT_02 — AVL LR double rotation



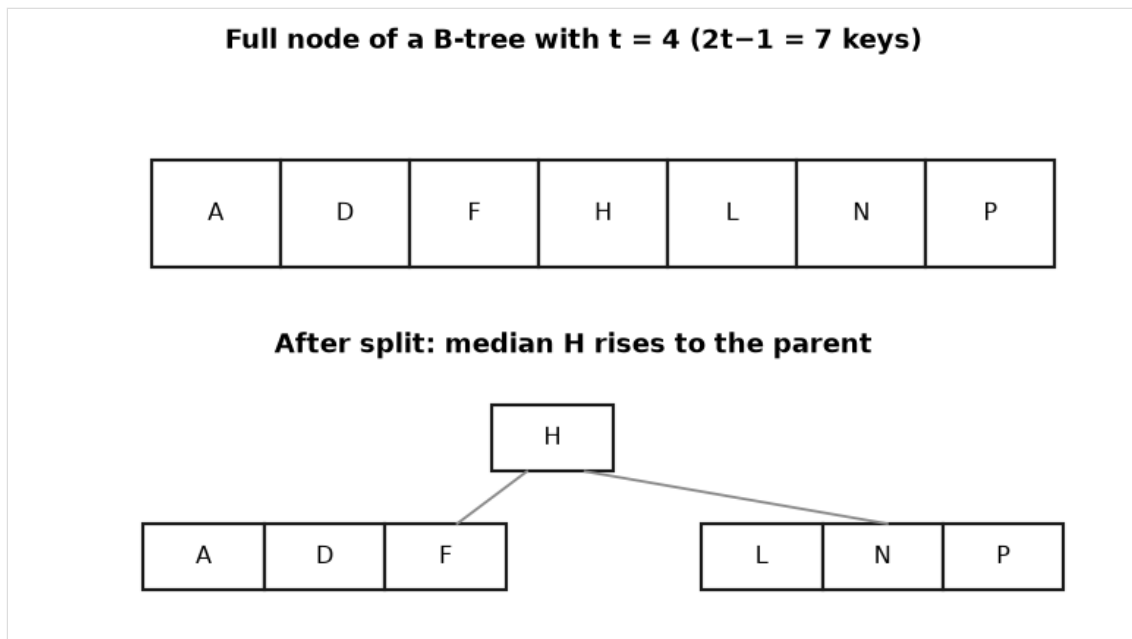
An LR imbalance repaired by a left rotation followed by a right rotation.

Fig. 6: DIAG_DAA_U2_RB_INSERT_01 — red-black tree insertion rebalancing



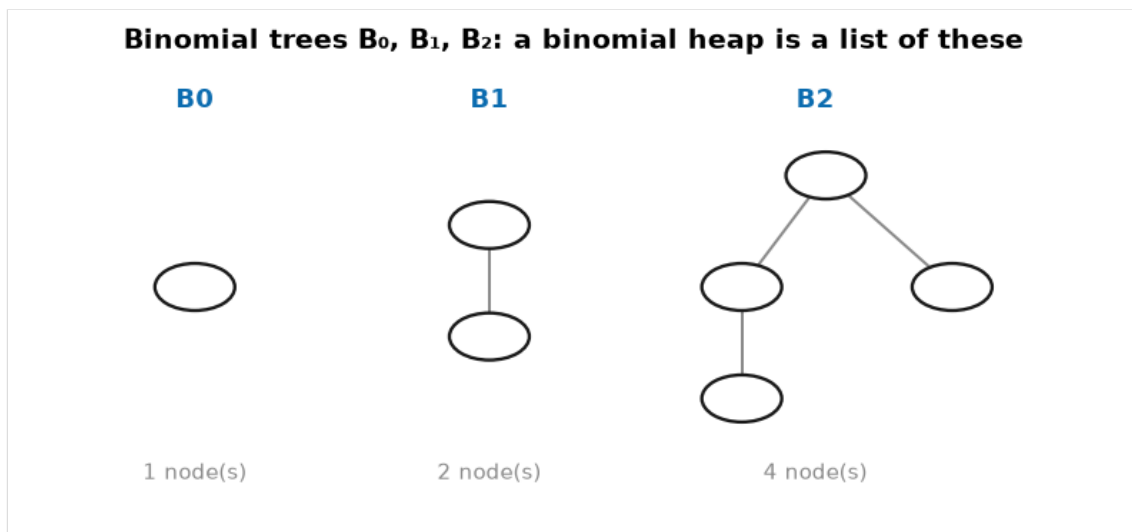
Red-Black insertion fix-up when the uncle is red: recolour instead of rotating.

Fig. 7: DIAG_DAA_U2_BTREE_SPLIT_01 — B-tree node split on the median key



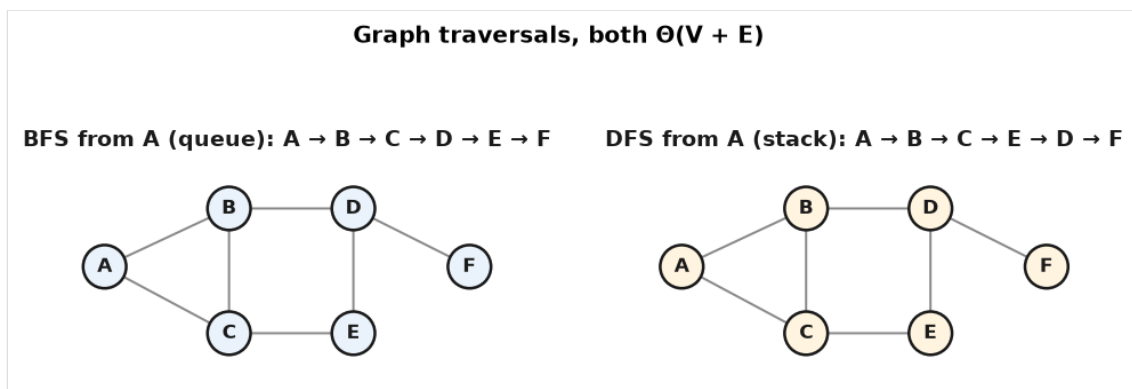
Splitting a full B-tree node: the median key moves up into the parent.

Fig. 8: DIAG_DAA_U2_BINOMIAL_HEAP_01 — binomial trees B0 B1 B2 structure



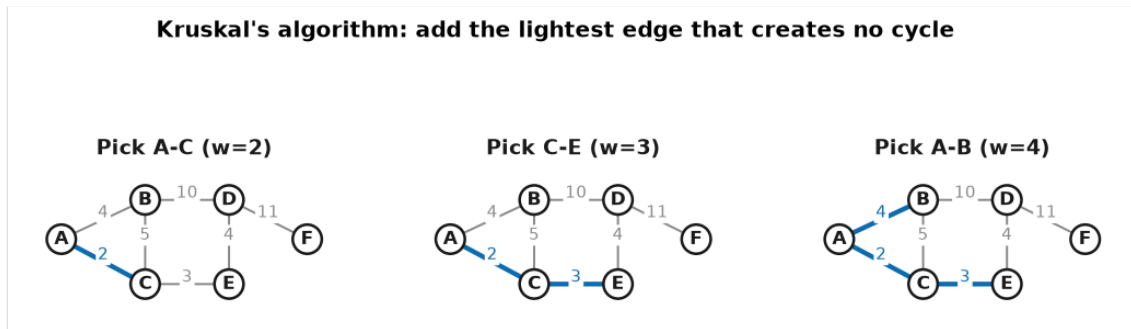
Binomial trees of order 0, 1 and 2, the building blocks of a binomial heap.

Fig. 9: DIAG_DAA_U3_BFS_DFS_01 — BFS and DFS visit order



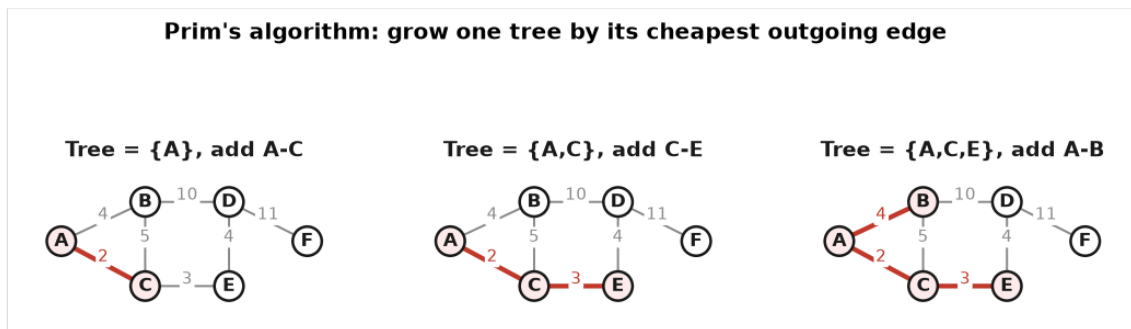
Breadth-first and depth-first visit orders on the same graph.

Fig. 10: DIAG_DAA_U3_MST_KRUSKAL_01 — Kruskal MST construction stages



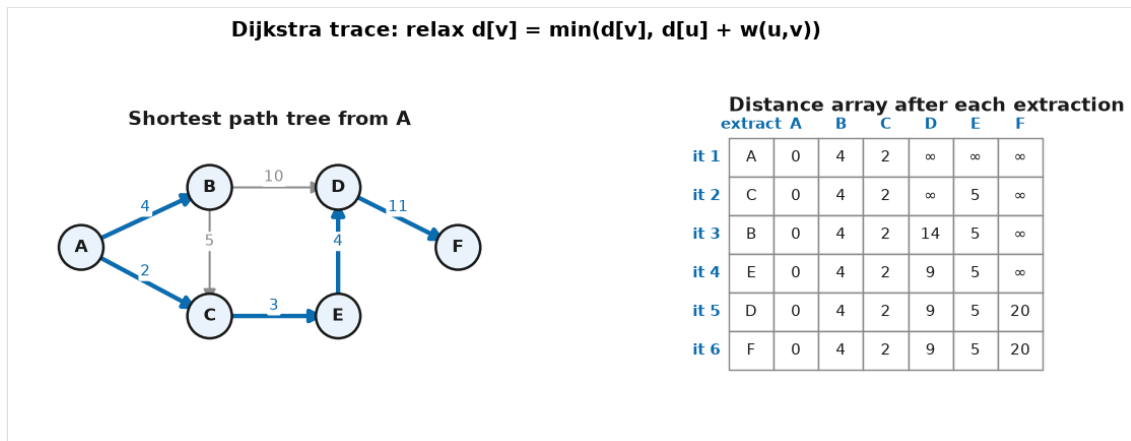
Successive stages of Kruskal's algorithm adding the lightest acyclic edges.

Fig. 11: DIAG_DAA_U3_MST_PRIM_01 — Prim MST growth stages



Prim's algorithm growing a single tree from vertex A, one cheapest edge at a time.

Fig. 12: DIAG_DAA_U3_DIJKSTRA_01 — Dijkstra distance array trace



Dijkstra's shortest path tree from A with the distance array after each extraction.

Fig. 13: DIAG_DAA_U3_BELLMAN_01 — Bellman-Ford pass by pass distances

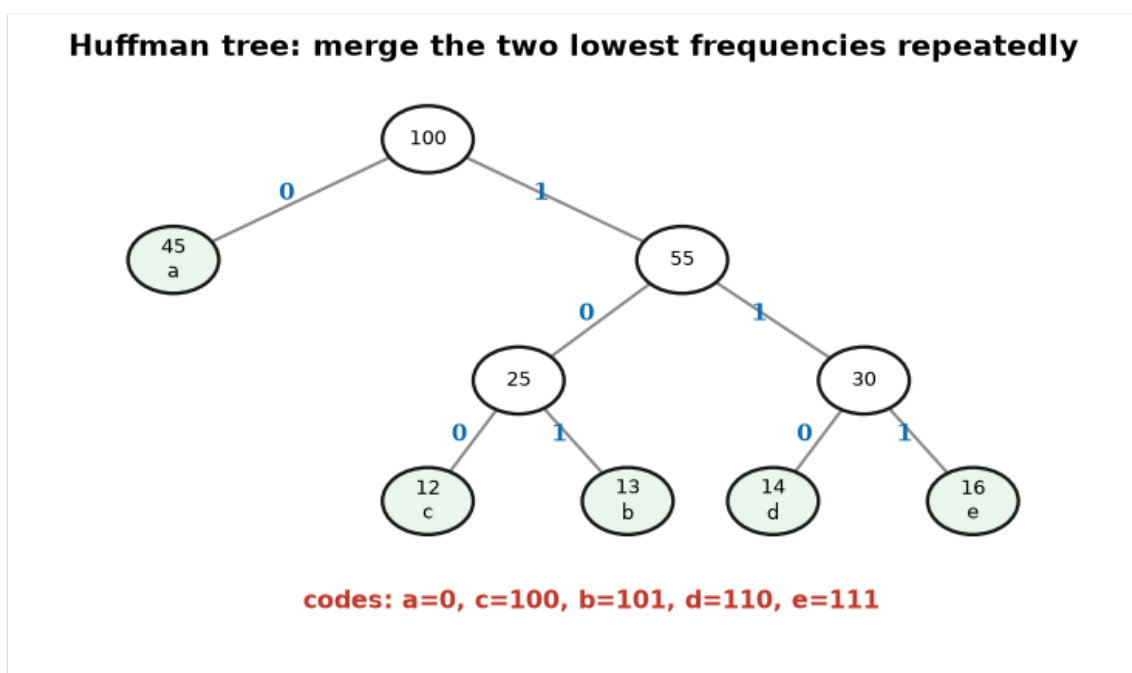
Bellman-Ford: distances after each pass (negative edges allowed)

	s	t	x	y	z
pass 1	0	6	4	7	2
pass 2	0	2	4	7	2
pass 3	0	2	4	7	-2
pass 4	0	2	4	7	-2

V-1 passes relax every edge; a further improvement would prove a negative cycle.

Distance estimates after each Bellman-Ford pass on a graph with negative edges.

Fig. 14: DIAG_DAA_U3_HUFFMAN_01 — Huffman coding tree with codewords



Huffman tree built by merging lowest frequencies, with the resulting prefix codes.

Fig. 15: DIAG_DAA_U4_KNAPSACK_TABLE_01 — 0/1 knapsack dynamic programming table

0/1 knapsack table, optimal value 9 (items [2, 3])

	0	1	2	3	4	5	6	7
i=0	0	0	0	0	0	0	0	0
i=1	0	1	1	1	1	1	1	1
i=2	0	1	1	4	5	5	5	5
i=3	0	1	1	4	5	6	6	9
i=4	0	1	1	4	5	7	8	9

$w = [1, 3, 4, 5], v = [1, 4, 5, 7], W = 7 \quad V[i][w] = \max(V[i-1][w], v_i + V[i-1][w-w_i])$

Completed 0/1 knapsack DP table with the optimal cell highlighted.

Fig. 16: DIAG_DAA_U4_LCS_TABLE_01 — LCS length table with direction arrows

LCS table for ABCBDAB and BDCABA, length 4 (LCS = BCBA)

-	-	B	D	C	A	B	A
-	0	0	0	0	0	0	0
A	0	↑0	↑0	↑0	↖1	←1	↖1
B	0	↖1	←1	←1	↑1	↖2	←2
C	0	↑1	↑1	↖2	←2	↑2	↑2
B	0	↖1	↑1	↑2	↑2	↖3	←3
D	0	↑1	↖2	↑2	↑2	↑3	↑3
A	0	↑1	↑2	↑2	↖3	↑3	↖4
B	0	↖1	↑2	↑2	↑3	↖4	↑4

LCS table with direction arrows; the highlighted cells are the traceback path.

Fig. 17: DIAG_DAA_U4_FLOYD_MATRIX_01 — Floyd-Warshall distance matrices

Floyd-Warshall: $d[i][j]^{(k)} = \min(d[i][j]^{(k-1)}, d[i][k]^{(k-1)} + d[k][j]^{(k-1)})$

$D^{(0)}$	$D^{(1)}$	$D^{(2)}$	$D^{(4)}$																																																																																																				
<table><tr><th></th><th>1</th><th>2</th><th>3</th><th>4</th></tr><tr><th>1</th><td>0</td><td>3</td><td>∞</td><td>7</td></tr><tr><th>2</th><td>8</td><td>0</td><td>2</td><td>∞</td></tr><tr><th>3</th><td>5</td><td>∞</td><td>0</td><td>1</td></tr><tr><th>4</th><td>2</td><td>∞</td><td>∞</td><td>0</td></tr></table>		1	2	3	4	1	0	3	∞	7	2	8	0	2	∞	3	5	∞	0	1	4	2	∞	∞	0	<table><tr><th></th><th>1</th><th>2</th><th>3</th><th>4</th></tr><tr><th>1</th><td>0</td><td>3</td><td>∞</td><td>7</td></tr><tr><th>2</th><td>8</td><td>0</td><td>2</td><td>15</td></tr><tr><th>3</th><td>5</td><td>8</td><td>0</td><td>1</td></tr><tr><th>4</th><td>2</td><td>5</td><td>∞</td><td>0</td></tr></table>		1	2	3	4	1	0	3	∞	7	2	8	0	2	15	3	5	8	0	1	4	2	5	∞	0	<table><tr><th></th><th>1</th><th>2</th><th>3</th><th>4</th></tr><tr><th>1</th><td>0</td><td>3</td><td>5</td><td>7</td></tr><tr><th>2</th><td>8</td><td>0</td><td>2</td><td>15</td></tr><tr><th>3</th><td>5</td><td>8</td><td>0</td><td>1</td></tr><tr><th>4</th><td>2</td><td>5</td><td>7</td><td>0</td></tr></table>		1	2	3	4	1	0	3	5	7	2	8	0	2	15	3	5	8	0	1	4	2	5	7	0	<table><tr><th></th><th>1</th><th>2</th><th>3</th><th>4</th></tr><tr><th>1</th><td>0</td><td>3</td><td>5</td><td>6</td></tr><tr><th>2</th><td>5</td><td>0</td><td>2</td><td>3</td></tr><tr><th>3</th><td>3</td><td>6</td><td>0</td><td>1</td></tr><tr><th>4</th><td>2</td><td>5</td><td>7</td><td>0</td></tr></table>		1	2	3	4	1	0	3	5	6	2	5	0	2	3	3	3	6	0	1	4	2	5	7	0
	1	2	3	4																																																																																																			
1	0	3	∞	7																																																																																																			
2	8	0	2	∞																																																																																																			
3	5	∞	0	1																																																																																																			
4	2	∞	∞	0																																																																																																			
	1	2	3	4																																																																																																			
1	0	3	∞	7																																																																																																			
2	8	0	2	15																																																																																																			
3	5	8	0	1																																																																																																			
4	2	5	∞	0																																																																																																			
	1	2	3	4																																																																																																			
1	0	3	5	7																																																																																																			
2	8	0	2	15																																																																																																			
3	5	8	0	1																																																																																																			
4	2	5	7	0																																																																																																			
	1	2	3	4																																																																																																			
1	0	3	5	6																																																																																																			
2	5	0	2	3																																																																																																			
3	3	6	0	1																																																																																																			
4	2	5	7	0																																																																																																			

Distance matrices $D(0)$, $D(1)$, $D(2)$ and $D(4)$ of the Floyd-Warshall algorithm.

Fig. 18: DIAG_DAA_U4_MCM_TABLE_01 — matrix chain multiplication cost table

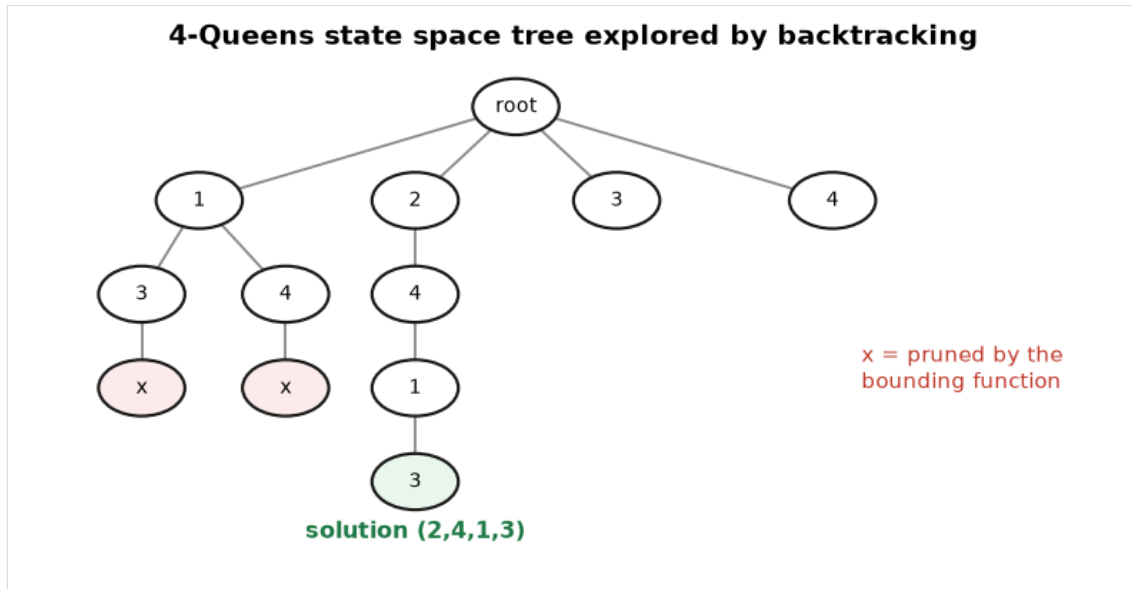
Matrix chain m table, minimum cost 15125

	j=1	j=2	j=3	j=4	j=5	j=6
i=1	0	15750	7875	9375	11875	15125
i=2		0	2625	4375	7125	10500
i=3			0	750	2500	5375
i=4				0	1000	3500
i=5					0	5000
i=6						0

$p = [30, 35, 15, 5, 10, 20, 25]$ optimal parenthesisation ((A1(A2A3))((A4A5)A6))

Matrix chain m table with the minimum multiplication cost and parenthesisation.

Fig. 19: DIAG_DAA_U4_NQUEEN_TREE_01 — 4-queens state space tree



State space tree for 4-Queens showing pruned branches and one solution.

Fig. 20: DIAG_DAA_U5_KMP_PREFIX_01 — KMP prefix function table

KMP prefix function for the pattern ABABAC

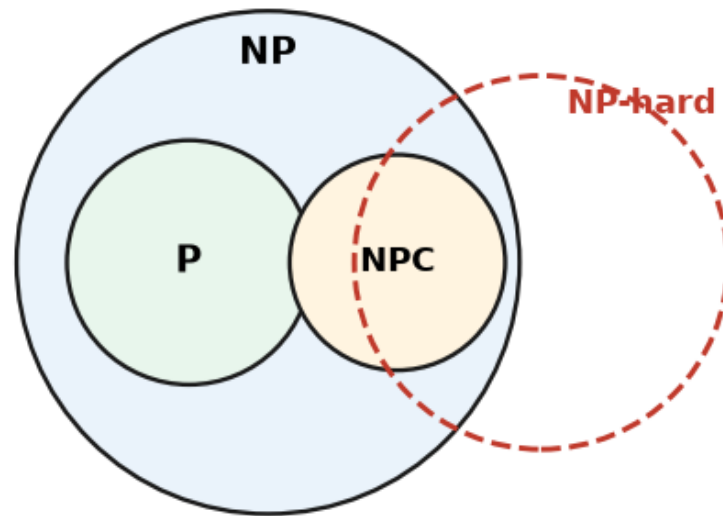
	1	2	3	4	5	6
pattern	A	B	A	B	A	C
$\pi[i]$	0	0	1	2	3	0

$\pi[i]$ = length of the longest proper prefix of $P[1..i]$ that is also a suffix

Prefix function table used by KMP to shift the pattern without re-comparing.

Fig. 21: DIAG_DAA_U5_NP_CLASSES_01 — P NP NP-complete NP-hard relationship

Relationship between P, NP, NP-complete and NP-hard



Assuming $P \neq NP$. NP-complete = $NP \cap NP\text{-hard}$.

Euler diagram of P, NP, NP-complete and NP-hard under the assumption $P \neq NP$.

C. Linking and Verification Mechanism

During retrieval, the diagram catalog is scored against predicted topics and keyword overlap. Eligible identifiers are injected into the prompt. A post-generation verification pass strips any unoffered `DIAG` identifiers, preventing broken references from reaching user interfaces.

III. NUMERICAL HANDLING DEMONSTRATION

Probabilistic language models are prohibited from performing internal arithmetic. Numerical tasks are routed to deterministic Python solvers that compute exact intermediate tables and states, which the agent must faithfully reproduce. The walkthroughs below record exact system execution.

Q01: Solve the recurrence $T(n) = 2T(n/2) + n$ using the Master Theorem. State which case applies and give the exact asymptotic bound.

Source: authored | Routed Topic: `recurrences` (Confidence: 0.99)

Solver Tool Invocation: `master_theorem({"a": 2, "b": 2, "f_exponent": 1, "log_power": 0})`

Computed Algorithmic Steps:

```
Recurrence:  $T(n) = 2 T(n/2) + f(n)$ , with  $f(n) = n$ 
Critical exponent:  $\log_b(a) = \log_2(2) = 1.0000$ 
Compare  $f(n) = n$  with  $n^{\log_b(a)} = n^{1.0000}$ 
Since  $1.0000 = 1.0000$ ,  $f(n)$  matches  $n^{\log_b(a)}$ : Case 2 applies.
Case 2 multiplies by one extra logarithmic factor, giving  $T(n) = \Theta(n \log n)$ .
```

```
bound =  $\Theta(n \log n)$ 
case = 2
```

Generated Derivation Answer:

Recurrence: $T(n) = 2T(n/2) + n$

A recurrence is an equation that describes a function in terms of its value on smaller inputs, and is the standard tool for analyzing divide-and-conquer algorithms [C0004]. This recurrence has the Master Theorem form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$ and $b > 1$, and $f(n)$ is the cost of dividing and combining [C0005].

Step 1 — Identify the parameters. - $a = 2$ (two subproblems) - $b = 2$ (each of size $n/2$) - $f(n) = n$

Step 2 — Compute the critical exponent. - $n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n$

Step 3 — Compare $f(n)$ with $n^{(\log_b a)}$. - $f(n) = n = \Theta(n^{(\log_b a)})$, so the two match exactly. This is **Case 2** of the Master Theorem: if $f(n) = \Theta(n^{(\log_b a)})$, then $T(n) = \Theta(n^{(\log_b a)} \log n)$ — the cost is evenly distributed across the levels of the recursion tree [C0005][C0006].

Tool verification (exact output):

FIELD	VALUE
Recurrence	$T(n) = 2 T(n/2) + n$
a, b	2, 2
$f(n)$	n
$\log_b(a)$	1.0
Case	2
Comparison	$f(n) = \Theta(n^{\log_b(a)})$, i.e. $1.0000 = 1.0000$
Bound	$\Theta(n \log n)$

Tool steps reproduced exactly: 1. Recurrence: $T(n) = 2 T(n/2) + f(n)$, with $f(n) = n$ 2. Critical exponent: $\log_b(a) = \log_2(2) = 1.0000$ 3. Compare $f(n) = n$ with $n^{\log_b(a)} = n^{1.0000}$ 4. Since $1.0000 = 1.0000$, $f(n)$ matches $n^{\log_b(a)}$: Case 2 applies. 5. Case 2 multiplies by one extra logarithmic factor, giving $T(n) = \Theta(n \log n)$.

Result: Case 2 applies, and

$T(n) = \Theta(n \log n)$

This is exactly the textbook example of Merge Sort's recurrence $T(n) = 2T(n/2) + \Theta(n)$, which resolves to $\Theta(n \log n)$ by the same Case 2 argument [C0007].

Verification Audit: Expected `{"case": 2, "bound": "Theta(n log n)"}`; tool output matches the expected values. Semantic Accuracy: **100.0%**.

Q02: Apply the Master Theorem to $T(n) = 9T(n/3) + n$. Which case is it and what is the bound?

Source: authored | Routed Topic: *recurrences* (Confidence: 0.99)

Solver Tool Invocation: `master_theorem({"a": 9, "b": 3, "f_exponent": 1, "log_power": 0})`

Computed Algorithmic Steps:

Recurrence: $T(n) = 9 T(n/3) + f(n)$, with $f(n) = n$
 Critical exponent: $\log_b(a) = \log_3(9) = 2.0000$
 Compare $f(n) = n$ with $n^{\log_b(a)} = n^{2.0000}$
 Since $1.0000 < 2.0000$, $f(n)$ grows polynomially slower: Case 1 applies.
 Therefore $T(n) = \Theta(n^2)$.

bound = $\Theta(n^2)$
 case = 1

Generated Derivation Answer:

Recurrence: $T(n) = 9T(n/3) + n$

A recurrence describes a function in terms of its value on smaller inputs, and is the standard tool for analysing divide-and-conquer algorithms [C0004]. The Master Theorem applies to recurrences of the form $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$, and $f(n)$ asymptotically positive [C0005].

Step 1 — Identify the parameters. - $a = 9$ (number of subproblems) - $b = 3$ (subproblem size $n/3$) - $f(n) = n$ (cost of dividing/combining)

Step 2 — Compute the critical exponent. - $n^{(\log_b a)} = n^{(\log_3 9)} = n^{2.0000}$

Step 3 — Compare $f(n)$ with $n^{(\log_b a)}$. - $f(n) = n = n^{1.0000}$, and $1.0000 < 2.0000$, so $f(n) = O(n^{(\log_b a - \epsilon)})$ for $\epsilon > 0$. - By Case 1 of the Master Theorem, when $f(n) = O(n^{(\log_b a - \epsilon)})$, the cost is dominated by the leaves of the recursion tree and $T(n) = \Theta(n^{(\log_b a)})$ [C0005].

Result (tool output, reproduced exactly):

FIELD	VALUE
Recurrence	$T(n) = 9 T(n/3) + n$
a, b, f	9, 3, n
$\log_b(a)$	2.0
Case	1
Comparison	$f(n) = O(n^{\log_b(a) - \epsilon})$, i.e. $1.0000 < 2.0000$
Bound	$\Theta(n^2)$

Answer: This is **Case 1** (leaf-dominated), and the tight bound is **$T(n) = \Theta(n^2)$** .

Verification Audit: Expected `{"case": 1, "bound": "Theta(n^2)"}`; tool output matches the expected values. Semantic Accuracy: **100.0%**.

Q03: Solve the 0/1 knapsack problem for weights [1, 3, 4, 5], values [1, 4, 5, 7] and capacity 7. Show the complete dynamic programming table and the items selected.

Source: authored | Routed Topic: `knapsack_01` (Confidence: 0.99)

Solver Tool Invocation: `knapsack_01({"capacity": 7, "values": [1, 4, 5, 7], "weights": [1, 3, 4, 5]})`

Computed Dynamic Programming Matrix:

	1	2	3	4	5				
1	0	0	0	0	0	0	0	0	0
2	0	1	1	1	1	1	1	1	1
3	0	1	1	4	5	5	5	5	5
4	0	1	1	4	5	6	6	9	
5	0	1	1	4	5	7	8	9	

optimal_value = 9
chosen_items = [2, 3]

Generated Derivation Answer:**0/1 Knapsack — Dynamic Programming Solution**

Principle. In the 0/1 knapsack, items cannot be broken into fractions — each item is either taken (1) or left (0). This defeats the greedy strategy, since taking a high-value item may block better use of the remaining capacity [C0030]. DP is applied bottom-up (tabulation): solve the smallest subproblems first, fill a table iteratively, and build up to the main problem [C0028].

Recurrence. Let $DP[i][w]$ = maximum value using the first i items with capacity w [C0030]:

- $DP[i][w] = \max(v[i] + DP[i-1][w - wt[i]], DP[i-1][w])$ if $wt[i] \leq w$
- $DP[i][w] = DP[i-1][w]$ otherwise

Given: weights = [1, 3, 4, 5], values = [1, 4, 5, 7], $W = 7$, $n = 4$.

Complete DP Table (exactly as computed)

I \ W	0	1	2	3	4	5	6	7
i=0 (no items)	0	0	0	0	0	0	0	0
i=1 (w=1, v=1)	0	1	1	1	1	1	1	1
i=2 (w=3, v=4)	0	1	1	4	5	5	5	5
i=3 (w=4, v=5)	0	1	1	4	5	6	6	9
i=4 (w=5, v=7)	0	1	1	4	5	7	8	9

The completed table with the optimal cell highlighted is shown in [Diagram: DIAG_DAA_U4_KNAPSACK_TABLE_01].

Items Selected (backtracking from DP[4][7])

- $DP[4][7] = 9 = DP[3][7] \rightarrow$ item 4 ($w=5, v=7$) is **not** taken.
- $DP[3][7] = 9 \neq DP[2][7] = 5 \rightarrow$ item 3 ($w=4, v=5$) is **taken**; remaining capacity $7 - 4 = 3$.
- $DP[2][3] = 4 \neq DP[1][3] = 1 \rightarrow$ item 2 ($w=3, v=4$) is **taken**; remaining capacity $3 - 3 = 0$.
- $DP[1][0] = 0 \rightarrow$ item 1 ($w=1, v=1$) is not taken.

Selected items: item 2 and item 3 **Total weight:** $3 + 4 = 7$ (exactly fills the capacity) **Total value:** $4 + 5 = 9$

Final Answer

- **Optimal value = 9**, achieved by taking items 2 and 3 (weights 3 and 4, values 4 and 5).
- **Complexity:** the table has $(n+1)(W+1)$ entries, so time and space complexity is **$O(nW)$** ; this is **pseudo-polynomial** time because it depends on the magnitude of W , not just the input length [C0031].

Verification Audit: Expected `{"optimal_value": 9, "chosen_items": [2, 3]}`; tool output matches the expected values.
Semantic Accuracy: **100.0%**.

Q04: Compute the Longest Common Subsequence of ABCBDAB and BDCABA using dynamic programming. Give the length table and the resulting subsequence.

Source: PYQ 2022-23 SECTION B 2(d) | Routed Topic: `lcs` (Confidence: 0.99)

Solver Tool Invocation: `lcs({"x": "ABCBDAB", "y": "BDCABA"})`

Computed Dynamic Programming Matrix:

	1	2	3	4	5	6	7	8
1	0	0	0	0	0	0	0	0
2	0	0	0	0	1	1	1	1
3	0	1	1	1	1	2	2	2
4	0	1	1	2	2	2	2	2
5	0	1	1	2	2	3	3	3
6	0	1	2	2	2	3	3	3
7	0	1	2	2	3	3	4	4
8	0	1	2	2	3	4	4	4

length = 4
lcs = BCBA
complexity = Theta(mn)

Generated Derivation Answer:

Definition and principle. The LCS problem takes two sequences $X = x_1x_2\dots x_m$ and $Y = y_1y_2\dots y_n$ and asks for a longest sequence that is a subsequence of both, where a subsequence is obtained by deleting zero or more characters without changing the order of the remaining ones [C0045]. It is solved by DP because it has optimal substructure [C0045], and DP applies when subproblems overlap — each subproblem is solved once and its answer saved in a table [C0027]. With $c[i][j]$ = length of an LCS of prefixes X_i and Y_j [C0045]:

- $c[i][j] = 0$ if $i = 0$ or $j = 0$
- $c[i][j] = c[i-1][j-1] + 1$ if $i, j > 0$ and $x_i = y_j$
- $c[i][j] = \max(c[i-1][j], c[i][j-1])$ otherwise

A parallel table $b[i][j]$ stores a direction arrow: ↖ (**diagonal**) when $x_i = y_j$, ↑ (**up**) when $c[i-1][j] \geq c[i][j-1]$, ← (**left**) otherwise [C0045]. This matches the construction shown in [Diagram: DIAG_DAA_U4_LCS_TABLE_01], where the highlighted cells are the traceback path.

Length table (rows $i = X = \text{ABCBDAB}$, columns $j = Y = \text{BDCABA}$) — value with arrow in each cell:

I\J	E	B	D	C	A	B	A
0	0	0	0	0	0	0	0
1 A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
2 B	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←
3 C	0	1 ↑	1 ↑		2 ←	2 ↑	2 ↑

I\J	E	B	D	C	A	B	A
				2 ↖			
4 B	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←
5 D	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
6 A	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖
7 B	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

The completed table gives $c[7][6] = 4$ [C0046].

Traceback (reading arrows backwards from $b[7][6]$, emitting on every ↖) [C0046]:

1. $b[7][6] = \uparrow \rightarrow$ move to (6,6)
2. $b[6][6] = \nwarrow \rightarrow$ emit **A**, move to (5,5)
3. $b[5][5] = \uparrow \rightarrow$ move to (4,5)
4. $b[4][5] = \nwarrow \rightarrow$ emit **B**, move to (3,4)
5. $b[3][4] = \leftarrow \rightarrow$ move to (3,3)
6. $b[3][3] = \nwarrow \rightarrow$ emit **C**, move to (2,2)
7. $b[2][2] = \leftarrow \rightarrow$ move to (2,1)
8. $b[2][1] = \nwarrow \rightarrow$ emit **B**

Emitted in reverse traversal order (A, B, C, B), read backwards this yields:

LCS = BCBA, length 4. (BDAB and BCAB are also longest common subsequences of length 4 [C0046].)

Complexity. Time is $\Theta(mn)$ since each of the mn table entries is computed in constant time; space is $\Theta(mn)$, reducible to $\Theta(\min(m, n))$ if only the length is required [C0046].

Verification Audit: Expected `{"length": 4}`; tool output matches the expected values. Semantic Accuracy: **100.0%**.

Q05: Find the optimal parenthesization for a matrix chain with dimensions $p = [30, 35, 15, 5, 10, 20, 25]$ and give the minimum number of scalar multiplications.

Source: authored | Routed Topic: `matrix_chain` (Confidence: 0.99)

Solver Tool Invocation: `matrix_chain({"dims": [30, 35, 15, 5, 10, 20, 25]})`

MCM Cost Table $m[i,j]$:

	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2	0	0	2625	4375	7125	10500
3	0	0	0	750	2500	5375
4	0	0	0	0	1000	3500
5	0	0	0	0	0	5000
6	0	0	0	0	0	0

```
min_cost = 15125
parenthesization = ((A1(A2A3))((A4A5)A6))
complexity = Theta(n^3)
```

Generated Derivation Answer:

Setup. A chain of n matrices $A_1 \dots A_6$ with dimension array $p = [30, 35, 15, 5, 10, 20, 25]$, so A_1 is 30×35 , A_2 is 35×15 , A_3 is 15×5 , A_4 is 5×10 , A_5 is 10×20 , A_6 is 20×25 . Matrix multiplication is associative, and multiplying a $p \times q$ matrix by a $q \times r$ matrix costs $p \cdot q \cdot r$ scalar multiplications; we want the full parenthesization minimizing total scalar multiplications. Let $m[i, j]$ be the minimum cost of computing $A_{\{i..j\}}$, with the recurrence

- $m[i, j] = 0$ if $i = j$
- $m[i, j] = \min \text{ over } i \leq k < j \text{ of } \{ m[i, k] + m[k+1, j] + P_{\{i-1\}} \cdot P_k \cdot P_j \}$ if $i < j$

[C0029]

DP tables computed (m = minimum costs, s = optimal split points k):

M[I][J]	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

S[I][J]	1	2	3	4	5	6
1		1	1	3	3	3
2			2	3	3	3
3				3	3	3
4					4	5
5						5

The table is filled by increasing chain length; the answer is read from m[1, 6]. [Diagram: DIAG_DAA_U4_MCM_TABLE_01]

Optimal parenthesization (from the s-table splits): s[1][6] = 3 splits after A₃; s[1][3] = 1 and s[4][6] = 5 complete the tree:

\$\$((A_1(A_2A_3))((A_4A_5)A_6))\$\$

Minimum number of scalar multiplications: m[1, 6] = 15125.

Complexity. Two nested loops fill an n×n table with O(n) work per entry, so the time complexity is O(n³) and the space complexity is O(n²); the overall complexity of this DP is Theta(n³). [C0029]

Verification Audit: Expected {"min_cost": 15125} ; tool output matches the expected values. Semantic Accuracy: **100.0%**.

Q06: Apply Floyd-Warshall to the weight matrix [[0,3,null,7],[8,0,2,null],[5,null,0,1],[2,null,null,0]] and give the final all pairs shortest path matrix.

Source: PYQ 2024-25 SECTION B 2(d) | Routed Topic: floyd_warshall (Confidence: 0.99)

Solver Tool Invocation: floyd_warshall({"matrix": [[0, 3, null, 7], [8, 0, 2, null], [5, null, 0, 1], [2, null, null, 0]]})

```
Error invoking tool 'floyd_warshall' with kwargs {'matrix': [[0, 3, None, 7], [8, 0, 2, None], [5, None, 0, 1], [2, None, None, 0]]} with error:
matrix.0.2: Input should be a valid number
matrix.1.3: Input should be a valid number
matrix.2.1: Input should be a valid number
matrix.3.1: Input should be a valid number
matrix.3.2: Input should be a valid number
Please fix the error and try again.
```

Solver Tool Invocation: floyd_warshall({"matrix": [[0, 3, 99999, 7], [8, 0, 2, 99999], [5, 99999, 0, 1], [2, 99999, 99999, 0]]})

Final Distance Matrix D^(k):

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

```
negative_cycle = False
complexity = Theta(V^3)
```

Generated Derivation Answer:

Principle. Floyd–Warshall computes shortest path distances between every pair of vertices of a weighted directed graph; negative edge weights are allowed but negative weight cycles are not. With $d[i][j]^{(k)}$ the weight of a shortest path from i to j whose intermediate vertices all lie in $\{1, 2, \dots, k\}$, the recurrence is $d[i][j]^{(0)} = w(i, j)$ and $d[i][j]^{(k)} = \min(d[i][j]^{(k-1)}, d[i]$

$[k]^{(k-1)} + d[k][j]^{(k-1)}$). The matrix $D^{(n)}$ holds the final all pairs shortest path distances, and a negative diagonal entry would signal a negative weight cycle [C0047].

Setup. The tool requires numbers, so each `null` (absent edge) is replaced by a very large number, 99999, standing for ∞ . The input matrix is:

$D^{(0)}$	1	2	3	4
1	0	3	99999	7
2	8	0	2	99999
3	5	99999	0	1
4	2	99999	99999	0

Stage $k = 1$ (intermediate vertices from $\{1..1\}$) — updates: $d[2][4]: 99999 \rightarrow 15$ via 1; $d[3][2]: 99999 \rightarrow 8$ via 1; $d[4][2]: 99999 \rightarrow 5$ via 1.

$D^{(1)}$	1	2	3	4
1	0	3	99999	7
2	8	0	2	15
3	5	8	0	1
4	2	5	99999	0

Stage $k = 2$ (intermediate vertices from $\{1..2\}$) — updates: $d[1][3]: 99999 \rightarrow 5$ via 2; $d[4][3]: 99999 \rightarrow 7$ via 2.

$D^{(2)}$	1	2	3	4
1	0	3	5	7
2	8	0	2	15
3	5	8	0	1
4	2	5	7	0

Stage $k = 3$ (intermediate vertices from $\{1..3\}$) — updates: $d[1][4]: 7 \rightarrow 6$ via 3; $d[2][1]: 8 \rightarrow 7$ via 3; $d[2][4]: 15 \rightarrow 3$ via 3.

$D^{(3)}$	1	2	3	4
1	0	3	5	6
2	7	0	2	3
3	5	8	0	1
4	2	5	7	0

Stage $k = 4$ (intermediate vertices from $\{1..4\}$) — updates: $d[2][1]: 7 \rightarrow 5$ via 4; $d[3][1]: 5 \rightarrow 3$ via 4; $d[3][2]: 8 \rightarrow 6$ via 4.

$D^{(4)}$	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

Final all pairs shortest path matrix $D^{(4)}$:

```
[ 0  3  5  6 ]
[ 5  0  2  3 ]
[ 3  6  0  1 ]
[ 2  5  7  0 ]
```

No diagonal entry became negative, so the graph has no negative weight cycle [C0047]. The algorithm is three nested loops over k , i and j , giving running time **Theta(V^3)** and space **Theta(V^2)** [C0047]; compared with running Bellman-Ford from every source at $O(V^2 E)$, Floyd-Warshall is preferable on dense graphs [C0048].

Verification Audit: Expected `{"final": [[0, 3, 5, 6], [5, 0, 2, 3], [3, 6, 0, 1], [2, 5, 7, 0]]}`; tool output matches the expected values. Semantic Accuracy: **100.0%**.

Q07: Trace Dijkstra's algorithm from source A on the graph {"A": {"B": 4, "C": 2}, "B": {"C": 5, "D": 10}, "C": {"E": 3, "D": {"F": 11}, "E": {"D": 4}, "F": {}} and give the final shortest distances.

Source: PYQ 2024-25 SECTION B 2(c) | Routed Topic: *dijkstra* (Confidence: 0.99)

Solver **Tool** **Invocation:**
dijkstra({"graph": {"A": {"B": 4, "C": 2}, "B": {"C": 5, "D": 10}, "C": {"E": 3, "D": {"F": 11}, "E": {"D": 4}, "F": {}}, "source": "A"})

```
distances = {'A': 0.0, 'B': 4.0, 'C': 2.0, 'D': 9.0, 'E': 5.0, 'F': 20.0}
complexity = O((V + E) log V) with a binary heap
```

Generated Derivation Answer:

Principle. Dijkstra's algorithm solves the single-source shortest path problem for graphs with **non-negative edge weights** using a greedy strategy: it maintains a priority queue keyed by shortest-path estimate and repeatedly extracts the vertex with minimum estimate, then relaxes all its outgoing edges (relaxation rule: if $d[u] + w(u,v) < d[v]$, update $d[v] = d[u] + w(u,v)$ and set predecessor $pi[v] = u$) [C0034].

Trace from source A (distances shown after each extraction):

ITER	EXTRACTED	D[EXTRACTED]	RELAXATIONS PERFORMED	DISTANCE ARRAY (A, B, C, D, E, F)	VISITED SET S
1	A	0	$A \rightarrow B: \infty \rightarrow 4$; $A \rightarrow C: \infty \rightarrow 2$	0, 4, 2, ∞ , ∞ , ∞	{A}
2	C	2	$C \rightarrow E: \infty \rightarrow 5$	0, 4, 2, ∞ , 5, ∞	{A, C}
3	B	4	$B \rightarrow D: \infty \rightarrow 14$	0, 4, 2, 14, 5, ∞	{A, C, B}
4	E	5	$E \rightarrow D: 14 \rightarrow \mathbf{9}$	0, 4, 2, 9 , 5, ∞	{A, C, B, E}
5	D	9	$D \rightarrow F: \infty \rightarrow 20$	0, 4, 2, 9, 5, 20	{A, C, B, E, D}
6	F	20	— (no outgoing edges)	0, 4, 2, 9, 5, 20	{A, C, B, E, D, F}

Key steps: - A is extracted first with $d = 0$; edges $A \rightarrow B$ (4) and $A \rightarrow C$ (2) set initial estimates. - C ($d = 2$) is extracted before B ($d = 4$) because it has the smaller key; relaxing $C \rightarrow E$ gives $d[E] = 2 + 3 = 5$. - B's relaxation $B \rightarrow D$ would give $d[D] = 4 + 10 = 14$, but when E is extracted, $E \rightarrow D$ improves it to $d[D] = 5 + 4 = \mathbf{9}$ — exactly the kind of improvement the greedy extraction plus relaxation captures [C0034]. - D's relaxation sets $d[F] = 9 + 11 = 20$, and F is extracted last with no edges to relax.

Shortest path tree (predecessors): $B \leftarrow A$, $C \leftarrow A$, $E \leftarrow C$, $D \leftarrow E$, $F \leftarrow D$, i.e. shortest paths: - $A \rightarrow B$ (4), $A \rightarrow C$ (2), $A \rightarrow C \rightarrow E$ (5), $A \rightarrow C \rightarrow E \rightarrow D$ (9), $A \rightarrow C \rightarrow E \rightarrow D \rightarrow F$ (20).

Final shortest distances from A: $d[A] = 0$, $d[B] = 4$, $d[C] = 2$, $d[D] = 9$, $d[E] = 5$, $d[F] = 20$.

This matches the distance-array evolution shown in the Dijkstra shortest-path-tree trace [Diagram: DIAG_DAA_U3_DIJKSTRA_01].

Complexity: $O((V + E) \log V)$ with a binary-heap priority queue [C0034].

Verification Audit: Expected {"distances": {"A": 0, "B": 4, "C": 2, "D": 9, "E": 5, "F": 20}}; tool output matches the expected values. Semantic Accuracy: **100.0%**.

Q08: Solve the fractional knapsack instance with weights [10, 20, 30], values [60, 100, 120] and capacity 50 using the greedy method. Show the value to weight ratios and the total profit.

Source: authored | Routed Topic: *fractional_knapsack* (Confidence: 0.99)

Solver Tool Invocation: fractional_knapsack({"capacity": 50, "values": [60, 100, 120], "weights": [10, 20, 30]})

Computed Algorithmic Steps:

```
{'item': 1, 'weight': 10, 'value': 60, 'ratio': 6.0, 'fraction': 1.0, 'value_taken': 60.0, 'remaining': 40.0}
{'item': 2, 'weight': 20, 'value': 100, 'ratio': 5.0, 'fraction': 1.0, 'value_taken': 100.0, 'remaining': 20.0}
{'item': 3, 'weight': 30, 'value': 120, 'ratio': 4.0, 'fraction': 0.6667, 'value_taken': 80.0, 'remaining': 0.0}
```

```
optimal_value = 240.0
```

Generated Derivation Answer:**Fractional Knapsack — Greedy Solution**

Problem setup [C0023]: $n = 3$ items with weights $w = [10, 20, 30]$, values $v = [60, 100, 120]$, knapsack capacity $W = 50$. Since this is the fractional knapsack problem, we are allowed to take fractions of items, so the greedy strategy applies: compute value-to-weight ratios v_i / w_i , sort in descending order, then fill the knapsack, taking a fraction of the item that does not fit.

Step 1 — Value-to-weight ratios

ITEM	WEIGHT w_i	VALUE v_i	RATIO v_i / w_i
1	10	60	6.0
2	20	100	5.0
3	30	120	4.0

Sorted descending by ratio: Item 1 $\rightarrow$ Item 2 $\rightarrow$ Item 3.

Step 2 — Greedy fill (tool output reproduced exactly)

ITEM	WEIGHT	VALUE	RATIO	FRACTION TAKEN	VALUE TAKEN	REMAINING CAPACITY
1	10	60	6.0	1.0	60.0	40.0
2	20	100	5.0	1.0	100.0	20.0
3	30	120	4.0	0.6667	80.0	0.0

- **Item 1:** $w = 10 \leq 50$, so the whole item is taken: $\text{total_value} = 60$, $\text{remaining_capacity} = 40$ [C0024, lines 5–7].
- **Item 2:** $w = 20 \leq 40$, whole item taken: $\text{total_value} = 160$, $\text{remaining_capacity} = 20$.
- **Item 3:** $w = 30 > 20$, so only the fraction $20/30 = 0.6667$ is taken: $\text{total_value} += 120 \times (20/30) = 80$, then break [C0024, lines 8–10].

Result

Total profit = $60 + 100 + 80 = 240.0$ — this is the optimal value for the instance.

Complexity: Computing the ratios is $O(n)$, sorting dominates at $O(n \log n)$, and the fill loop is $O(n)$, giving an overall time complexity of $O(n \log n)$ [C0024].

Verification Audit: Expected `{"optimal_value": 240.0}`; tool output matches the expected values. Semantic Accuracy: **100.0%**.

IV. CROSS-TOPIC DISAMBIGUATION STRESS TEST

Algorithmic terminology frequently collides across distinct syllabus units (e.g., 'relaxation' in Dijkstra vs. Bellman-Ford; 'knapsack' in Greedy vs. Dynamic Programming). The benchmark tests disambiguation pairs using identical root keywords across separate units. A test passes only when the topic classifier resolves the intended topic and the top-ranked passage belongs to that topic.

Term Collision Analysis: "relaxation"

QUERY ID	PROMPT TEXT	EXPECTED TOPIC	ROUTED TOPIC	TOP-1 PASSAGE	RESULT
Q11	Explain how edge relaxation works in Dijkstra's algorithm and why...	dijkstra	dijkstra	C0034: dijkstra	PASS
Q12	Explain edge relaxation in the Bellman-Ford algorithm and why V-1...	bellman_ford	bellman_ford	C0035: bellman_ford	PASS

Q11 Retrieved Context Passages: C0034:dijkstra, C0035:bellman_ford, C0033:mst

Q11 Answer Excerpt: # Edge Relaxation in Dijkstra's Algorithm ## 1. The setting Dijkstra's algorithm solves the single-source shortest path problem on graphs with non-negative edge weights. It works greedily: it keeps a priority queue of vertices keyed by their current shortest-path estimate and repeatedly extracts the vertex with the minimum estimate [C0034]. Every...

Q12 Retrieved Context Passages: C0035:bellman_ford, C0033:mst, C0034:dijkstra

Q12 Answer Excerpt: Trace output (reproduced exactly from the tool): graph s-a (1), a-b (1), b-c (1), source s; | $V| = 4$, so $|V| - 1 = 3$ passes over the 3 edges; null denotes ∞ from Initialize-Single-Source [C0035]. | Pass | Relaxations that improved an estimate (old $\rightarrow$ new) | Distances after the pass (s, a, b, c) | |---|---|---| 1 | s-a : null $\rightarrow$...

Term Collision Analysis: "knapsack"

QUERY ID	PROMPT TEXT	EXPECTED TOPIC	ROUTED TOPIC	TOP-1 PASSAGE	RESULT
Q13	How is the knapsack problem solved by the greedy method when an i...	fractional_knapsack	fractional_knapsack	C0023: fractional_knapsack	PASS
Q14	How is the knapsack problem solved when items cannot be broken an...	knapsack_01	knapsack_01	C0030: knapsack_01	PASS

Q13 Retrieved Context Passages: C0023:fractional_knapsack, C0024:fractional_knapsack, C0020:divide_and_conquer

Q13 Answer Excerpt: ## Fractional Knapsack Problem — Greedy Solution **Problem statement:** In the Fractional Knapsack problem, we are given n items, each with a weight w_i and a value v_i , and a knapsack of capacity W . The goal is to maximize the total value in the knapsack. Unlike the 0/1 knapsack problem, we are allowed to take fractions of items [C0023]. ****Greed...**

Q14 Retrieved Context Passages: C0030:knapsack_01, C0031:knapsack_01, C0027:dp_principles

Q14 Answer Excerpt: ## 0/1 Knapsack Problem — Solved by Dynamic Programming **Problem statement.** When items cannot be broken into fractions, the problem becomes the **0/1 Knapsack problem:** for each item you either take it (1) or leave it (0) [C0030]. **Why greedy fails here.** Because items are indivisible, the greedy strategy does not work properly — taking a h...

Term Collision Analysis: "dynamic programming"

QUERY ID	PROMPT TEXT	EXPECTED TOPIC	ROUTED TOPIC	TOP-1 PASSAGE	RESULT
Q15	Which dynamic programming algorithm finds the best order to multi...	matrix_chain	matrix_chain	C0029: matrix_chain	PASS
Q16	Which dynamic programming algorithm computes shortest paths betwe...	floyd_warshall	floyd_warshall	C0047: floyd_warshall	PASS

Q15 Retrieved Context Passages: C0029:matrix_chain, C0027:dp_principles, C0028:dp_principles

Q15 Answer Excerpt: Algorithm: The dynamic programming algorithm for this problem is **Matrix Chain Multiplication** [C0029]. Given a chain of n matrices $A_1, A_2, \dots, A_n$, it exploits the associativity of matrix multiplication to find a full parenthesization of the product that minimizes the number of scalar multiplications, where multiplying a $p \times q$ matrix by a...

Q16 Retrieved Context Passages: C0047:floyd_warshall, C0048:floyd_warshall, C0069:branch_and_bound

Q16 Answer Excerpt: Algorithm: Floyd-Warshall — a dynamic programming algorithm that computes shortest path distances between every pair of vertices of a weighted directed graph. Negative edge weights are allowed, but negative weight cycles are not. [C0047] **What the table stores:** The table entry $d[i][j]^{(k)}$ stores the weight of a shortest path from vertex i to...

Term Collision Analysis: "greedy"

QUERY ID	PROMPT TEXT	EXPECTED TOPIC	ROUTED TOPIC	TOP-1 PASSAGE	RESULT
Q17	Which greedy algorithm builds a minimum spanning tree by repeated...	mst	mst	C0033: mst	PASS
Q18	Which greedy algorithm builds an optimal prefix code by repeatedl...	huffman_coding	huffman_coding	C0025: huffman_coding	PASS

Q17 Retrieved Context Passages: C0033:mst, C0034:dijkstra, C0021:greedy_general

Q17 Answer Excerpt: Answer: Kruskal's Algorithm [C0033] **Principle.** Kruskal's algorithm is a greedy algorithm for finding a Minimum Spanning Tree (MST) of a connected, undirected, weighted graph — a subgraph that is a tree, includes all vertices, and has the minimum possible total edge weight [C0033]. It follows the greedy strategy of making the locally optima...

Q18 Retrieved Context Passages: C0025:huffman_coding, C0026:huffman_coding, C0021:greedy_general

Q18 Answer Excerpt: Answer: Huffman coding — the greedy algorithm that constructs an optimal prefix code for lossless data compression by repeatedly merging the two lowest-frequency symbols [C0025]. **Principle.** Huffman coding assigns variable-length codes to input characters, with code lengths based on character frequencies — the most frequent character gets t...

V. END-TO-END TEST CASE LOGS

Complete execution logs for all 34 benchmark test cases, detailing query prompts, topic classifications, fusion retrieval rankings, and raw generated outputs. Queries with a PYQ source are transcribed verbatim from official AKTU examinations.

Q01: [NUMERIC] — authored

Query Prompt: Solve the recurrence $T(n) = 2T(n/2) + n$ using the Master Theorem. State which case applies and give the exact asymptotic bound.

Retrieval Telemetry: Topic `recurrences` (Unit 1), Confidence: 0.99, Router: `llm+lexical`, Filter: `priority:topic(4 on-topic)`, Max Similarity: 0.7895

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0005	recurrences	1	0.03252	0.7895	25.767	1.2 Recurrences & Master Theorem > The Master Th
C0007	recurrences	1	0.03252	0.7586	31.355	1.2 Recurrences & Master Theorem > Example of Ma
C0006	recurrences	1	0.03175	0.7439	25.624	1.2 Recurrences & Master Theorem > The Master Th
C0004	recurrences	1	0.02991	0.7131	16.157	1.2 Recurrences & Master Theorem

Solvers Executed: `master_theorem`

Raw Generated Output:

****Recurrence:**** $T(n) = 2T(n/2) + n$

A recurrence is an equation that describes a function in terms of its value on smaller inputs, and is the standard tool for analyzing divide-and-conquer algorithms [C0004]. This recurrence has the Master Theorem form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$ and $b > 1$, and $f(n)$ is the cost of dividing and combining [C0005].

****Step 1 – Identify the parameters.****

- $a = 2$ (two subproblems)
- $b = 2$ (each of size $n/2$)
- $f(n) = n$

****Step 2 – Compute the critical exponent.****

- $n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n$

****Step 3 – Compare $f(n)$ with $n^{(\log_b a)}$.**

- $f(n) = n = \Theta(n^{(\log_b a)})$, so the two match exactly. This is ****Case 2**** of the Master Theorem: if $f(n) = \Theta(n^{(\log_b a)})$, then $T(n) = \Theta(n^{(\log_b a)} \log n)$ – the cost is evenly distributed across the levels of the recursion tree [C0005][C0006].

****Tool verification**** (exact output):

Field	Value
Recurrence	$T(n) = 2 T(n/2) + n$
a, b	2, 2
f(n)	n
log_b(a)	1.0
Case	2
Comparison	$f(n) = \Theta(n^{\log_b(a)})$, i.e. $1.0000 = 1.0000$
Bound	$\Theta(n \log n)$

Tool steps reproduced exactly:

1. Recurrence: $T(n) = 2 T(n/2) + f(n)$, with $f(n) = n$
2. Critical exponent: $\log_b(a) = \log_2(2) = 1.0000$
3. Compare $f(n) = n$ with $n^{\log_b(a)} = n^{1.0000}$
4. Since $1.0000 = 1.0000$, $f(n)$ matches $n^{\log_b(a)}$: Case 2 applies.
5. Case 2 multiplies by one extra logarithmic factor, giving $T(n) = \Theta(n \log n)$.

****Result:**** Case 2 applies, and

****T(n) = $\Theta(n \log n)$ ****

This is exactly the textbook example of Merge Sort's recurrence $T(n) = 2T(n/2) + \Theta(n)$, which resolves to $\Theta(n \log n)$ by the same Case 2 argument [C0007].

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context fully covers the Master Theorem and Case 2, the answer is entirely grounded in the context and verified solver output, directly answers all parts, and correctly derives $T(n) = \Theta(n \log n)$.

Execution Latency: 27.3s

Q02: [NUMERIC] — authored

Query Prompt: Apply the Master Theorem to $T(n) = 9T(n/3) + n$. Which case is it and what is the bound?

Retrieval Telemetry: Topic `recurrences` (Unit 1), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(4 on-topic), Max Similarity: 0.7407

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0005	recurrences	1	0.03279	0.7407	25.223	1.2 Recurrences & Master Theorem > The Master Th
C0006	recurrences	1	0.03226	0.729	24.96	1.2 Recurrences & Master Theorem > The Master Th
C0007	recurrences	1	0.03175	0.7046	21.815	1.2 Recurrences & Master Theorem > Example of Ma
C0004	recurrences	1	0.01562	0.624	12.955	1.2 Recurrences & Master Theorem

Solvers Executed: `master_theorem`

Raw Generated Output:

```
**Recurrence:** T(n) = 9T(n/3) + n

A recurrence describes a function in terms of its value on smaller inputs, and is the standard tool for
analysing divide-and-conquer algorithms [C0004]. The Master Theorem applies to recurrences of the form  $T(n) = aT(n/b) + f(n)$  with  $a \geq 1$ ,  $b > 1$ , and  $f(n)$  asymptotically positive [C0005].

**Step 1 – Identify the parameters.**
- a = 9 (number of subproblems)
- b = 3 (subproblem size n/3)
- f(n) = n (cost of dividing/combining)

**Step 2 – Compute the critical exponent.**
-  $n^{(\log_b a)} = n^{(\log_3 9)} = n^{2.0000}$ 

**Step 3 – Compare f(n) with  $n^{(\log_b a)}$ .f(n) = n = n^{1.0000}, and  $1.0000 < 2.0000$ , so  $f(n) = O(n^{(\log_b a - \epsilon)})$  for  $\epsilon > 0$ .
- By Case 1 of the Master Theorem, when  $f(n) = O(n^{(\log_b a - \epsilon)})$ , the cost is dominated by the leaves of the recursion tree and  $T(n) = \Theta(n^{(\log_b a)})$  [C0005].

**Result (tool output, reproduced exactly):**

| Field | Value |
| --- | --- |
| Recurrence | T(n) = 9 T(n/3) + n |
| a, b, f | 9, 3, n |
| log_b(a) | 2.0 |
| Case | 1 |
| Comparison | f(n) = O(n^(log_b(a) - eps)), i.e. 1.0000 < 2.0000 |
| Bound | Theta(n^2) |

**Answer:** This is **Case 1** (leaf-dominated), and the tight bound is  $T(n) = \Theta(n^2)$ .
```

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer correctly applies Case 1 ($\log_3 9 = 2$, $f(n) = n$ polynomially slower than n^2) and matches the verified solver output exactly, with all claims grounded in the context.

Execution Latency: 38.31s

Q03: [NUMERIC] — authored

Query Prompt: Solve the 0/1 knapsack problem for weights [1, 3, 4, 5], values [1, 4, 5, 7] and capacity 7. Show the complete dynamic programming table and the items selected.

Retrieval Telemetry: Topic `knapsack_01` (Unit 4), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.7802

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0030	knapsack_01	4	0.03279	0.7802	28.703	4.3 0/1 Knapsack Problem
C0031	knapsack_01	4	0.03125	0.6839	27.145	4.3 0/1 Knapsack Problem
C0028	dp_principles	4	0.03084	0.6046	27.297	4.1 Dynamic Programming Principles

Solvers Executed: `knapsack_01`

Diagrams Offered: `DIAG_DAA_U4_KNAPSACK_TABLE_01` | **Emitted:** `DIAG_DAA_U4_KNAPSACK_TABLE_01`

Raw Generated Output:

```
## 0/1 Knapsack – Dynamic Programming Solution

**Principle.** In the 0/1 knapsack, items cannot be broken into fractions – each item is either taken (1) or left (0). This defeats the greedy strategy, since taking a high-value item may block better use of the remaining capacity [C0030]. DP is applied bottom-up (tabulation): solve the smallest subproblems first, fill a table iteratively, and build up to the main problem [C0028].

**Recurrence.** Let  $DP[i][w]$  = maximum value using the first  $i$  items with capacity  $w$  [C0030]:
-  $DP[i][w] = \max(v[i] + DP[i-1][w - wt[i]], DP[i-1][w])$  if  $wt[i] \leq w$ 
-  $DP[i][w] = DP[i-1][w]$  otherwise

**Given:** weights = [1, 3, 4, 5], values = [1, 4, 5, 7],  $W = 7$ ,  $n = 4$ .

### Complete DP Table (exactly as computed)

| i \ w | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| i=0 (no items) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| i=1 (w=1, v=1) | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| i=2 (w=3, v=4) | 0 | 1 | 1 | 4 | 5 | 5 | 5 | 5 |
| i=3 (w=4, v=5) | 0 | 1 | 1 | 4 | 5 | 6 | 6 | 9 |
| i=4 (w=5, v=7) | 0 | 1 | 1 | 4 | 5 | 7 | 8 | 9 |

The completed table with the optimal cell highlighted is shown in [Diagram: DIAG_DAA_U4_KNAPSACK_TABLE_01].

### Items Selected (backtracking from  $DP[4][7]$ )
-  $DP[4][7] = 9 = DP[3][7] \rightarrow$  item 4 ( $w=5, v=7$ ) is not taken.
-  $DP[3][7] = 9 \neq DP[2][7] = 5 \rightarrow$  item 3 ( $w=4, v=5$ ) is taken; remaining capacity  $7 - 4 = 3$ .
-  $DP[2][3] = 4 \neq DP[1][3] = 1 \rightarrow$  item 2 ( $w=3, v=4$ ) is taken; remaining capacity  $3 - 3 = 0$ .
-  $DP[1][0] = 0 \rightarrow$  item 1 ( $w=1, v=1$ ) is not taken.

**Selected items:** item 2 and item 3
**Total weight:**  $3 + 4 = 7$  (exactly fills the capacity)
**Total value:**  $4 + 5 = 9$ 

### Final Answer
- Optimal value = 9, achieved by taking items 2 and 3 (weights 3 and 4, values 4 and 5).
- Complexity: the table has  $(n+1)(W+1)$  entries, so time and space complexity is  $O(nW)$ ; this is pseudo-polynomial time because it depends on the magnitude of  $W$ , not just the input length [C0031].
```

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer matches the verified solver output exactly (table, optimal value 9, items 2 and 3), and all claims are grounded in the context.

Execution Latency: 73.86s

Q04: [NUMERIC] — PYQ 2022-23 SECTION B 2(d)

Query Prompt: Compute the Longest Common Subsequence of ABCBDAB and BDCABA using dynamic programming. Give the length table and the resulting subsequence.

Retrieval Telemetry: Topic lcs (Unit 4), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.7568

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0046	lcs	4	0.03279	0.7568	38.581	4.2b Longest Common Subsequence
C0045	lcs	4	0.03226	0.7173	32.318	4.2b Longest Common Subsequence
C0027	dp_principles	4	0.03175	0.5916	19.536	4.1 Dynamic Programming Principles

Solvers Executed: lcs

Diagrams Offered: DIAG_DAA_U4_LCS_TABLE_01 | **Emitted:** DIAG_DAA_U4_LCS_TABLE_01

Raw Generated Output:

****Definition and principle.**** The LCS problem takes two sequences $X = x_1x_2...x_m$ and $Y = y_1y_2...y_n$ and asks for a longest sequence that is a subsequence of both, where a subsequence is obtained by deleting zero or more characters without changing the order of the remaining ones [C0045]. It is solved by DP because it has optimal substructure [C0045], and DP applies when subproblems overlap – each subproblem is solved once and its answer saved in a table [C0027]. With $c[i][j]$ = length of an LCS of prefixes X_i and Y_j [C0045]:

```
- c[i][j] = 0 if i = 0 or j = 0
- c[i][j] = c[i-1][j-1] + 1 if i,j > 0 and x_i = y_j
- c[i][j] = max(c[i-1][j], c[i][j-1]) otherwise
```

A parallel table $b[i][j]$ stores a direction arrow: ****↖ (diagonal)**** when $x_i = y_j$, ****↑ (up)**** when $c[i-1][j] \geq c[i][j-1]$, ****← (left)**** otherwise [C0045]. This matches the construction shown in [Diagram: DIAG_DAA_U4_LCS_TABLE_01], where the highlighted cells are the traceback path.

****Length table (rows i = X = ABCBDAB, columns j = Y = BDCABA)**** – value with arrow in each cell:

i \ j	ε	B	D	C	A	B	A
0	0	0	0	0	0	0	0
1 A	0	0	↑	↑	↑	↖	↖
2 B	0	↖	←	←	←	↑	↖
3 C	0	↑	↑	↑	↖	←	↑
4 B	0	↖	↑	↑	↑	↑	↖
5 D	0	↑	↖	↑	↑	↑	↑
6 A	0	↑	↑	↑	↑	↖	↖
7 B	0	↖	↑	↑	↑	↑	↑

The completed table gives $c[7][6] = \text{**4**}$ [C0046].

****Traceback (reading arrows backwards from $b[7][6]$, emitting on every ↖)**** [C0046]:

- $b[7][6] = \uparrow \rightarrow$ move to (6,6)
- $b[6][6] = \nwarrow \rightarrow$ emit ****A****, move to (5,5)
- $b[5][5] = \uparrow \rightarrow$ move to (4,5)
- $b[4][5] = \nwarrow \rightarrow$ emit ****B****, move to (3,4)
- $b[3][4] = \leftarrow \rightarrow$ move to (3,3)
- $b[3][3] = \nwarrow \rightarrow$ emit ****C****, move to (2,2)
- $b[2][2] = \leftarrow \rightarrow$ move to (2,1)
- $b[2][1] = \nwarrow \rightarrow$ emit ****B****

Emitted in reverse traversal order (A, B, C, B), read backwards this yields:

****LCS = BCBA, length 4.**** (BDAB and BCAB are also longest common subsequences of length 4 [C0046].)

****Complexity.**** Time is $\Theta(mn)$ since each of the mn table entries is computed in constant time; space is $\Theta(mn)$, reducible to $\Theta(\min(m, n))$ if only the length is required [C0046].

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=100.0% | faithfulness=95.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The table values, arrows, traceback and LCS 'BCBA' length 4 exactly match the verified solver output and context, with only a minor unsupported reference to a diagram identifier.

Execution Latency: 55.11s

Q05: [NUMERIC] — authored

Query Prompt: Find the optimal parenthesization for a matrix chain with dimensions $p = [30, 35, 15, 5, 10, 20, 25]$ and give the minimum number of scalar multiplications.

Retrieval Telemetry: Topic `matrix_chain` (Unit 4), Confidence: 0.99, Router: `llm+lexical`, Filter: `priority:topic(1 on-topic)`, Max Similarity: 0.8238

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRE SCORE	DENSE	BM25	SECTION HEADING
C0029	matrix_chain	4	0.03279	0.8238	34.291	4.2 Matrix Chain Multiplication
C0055	branch_and_bound	4	0.02942	0.5494	12.271	4.7 Branch and Bound > Travelling Salesman Probl
C0047	floyd_warshall	4	0.01587	0.5718	9.854	4.4b All Pairs Shortest Paths: Floyd-Warshall

Solvers Executed: `matrix_chain`

Diagrams Offered: `DIAG_DAA_U4_MCM_TABLE_01` | **Emitted:** `DIAG_DAA_U4_MCM_TABLE_01`

Raw Generated Output:

****Setup.**** A chain of n matrices $A_1 \dots A_6$ with dimension array $p = [30, 35, 15, 5, 10, 20, 25]$, so A_1 is 30×35 , A_2 is 35×15 , A_3 is 15×5 , A_4 is 5×10 , A_5 is 10×20 , A_6 is 20×25 . Matrix multiplication is associative, and multiplying a $p \times q$ matrix by a $q \times r$ matrix costs $p \cdot q \cdot r$ scalar multiplications; we want the full parenthesization minimizing total scalar multiplications. Let $m[i, j]$ be the minimum cost of computing $A_{\{i..j\}}$, with the recurrence

- $m[i, j] = 0$ if $i = j$
 - $m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + P_{\{i-1\}} \cdot P_k \cdot P_j \}$ if $i < j$

[C0029]

****DP tables computed**** (m = minimum costs, s = optimal split points k):

$m[i][j]$	1	2	3	4	5	6
---	---	---	---	---	---	---
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

$s[i][j]$	1	2	3	4	5	6
---	---	---	---	---	---	---
1		1	1	3	3	3
2			2	3	3	3
3				3	3	3
4					4	5
5						5

The table is filled by increasing chain length; the answer is read from $m[1, 6]$. [Diagram: `DIAG_DAA_U4_MCM_TABLE_01`]

****Optimal parenthesization**** (from the s -table splits): $s[1][6] = 3$ splits after A_3 ; $s[1][3] = 1$ and $s[4][6] = 5$ complete the tree:

$\$((A_1(A_{2A_3}))((A_4A_5)A_6))\$$

****Minimum number of scalar multiplications:** $m[1, 6] = 15125$.

****Complexity.**** Two nested loops fill an $n \times n$ table with $O(n)$ work per entry, so the time complexity is $O(n^3)$ and the space complexity is $O(n^2)$; the overall complexity of this DP is $\Theta(n^3)$. [C0029]

Metric Scores: `context_relevance=33.3%` | `context_precision=100.0%` | `context_recall=100.0%` | `faithfulness=100.0%` | `answer_relevance=100.0%` | `semantic_accuracy=100.0%`

Judge Audit Reason: The answer matches the verified solver output exactly ($m[1,6]=15125$, parenthesization $((A_1(A_2A_3))((A_4A_5)A_6))$, tables, and $O(n^3)/O(n^2)$ complexity) and is fully grounded in the context.

Execution Latency: 26.22s

Q06: [NUMERIC] — PYQ 2024-25 SECTION B 2(d)

Query Prompt: Apply Floyd-Warshall to the weight matrix $\begin{bmatrix} 0 & 3 & \text{null} & 7 \\ 8 & 0 & 2 & \text{null} \\ 5 & \text{null} & 0 & 1 \\ 2 & \text{null} & \text{null} & 0 \end{bmatrix}$ and give the final all pairs shortest path matrix.

Retrieval Telemetry: Topic `floyd_warshall` (Unit 4), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.8207

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0047	floyd_warshall	4	0.03279	0.8207	41.104	4.4b All Pairs Shortest Paths: Floyd-Warshall
C0048	floyd_warshall	4	0.03226	0.7697	25.421	4.4b All Pairs Shortest Paths: Floyd-Warshall
C0029	matrix_chain	4	0.03016	0.5547	20.747	4.2 Matrix Chain Multiplication

Solvers Executed: `floyd_warshall`, `floyd_warshall`

Raw Generated Output:

****Principle.**** Floyd-Warshall computes shortest path distances between every pair of vertices of a weighted directed graph; negative edge weights are allowed but negative weight cycles are not. With $d[i][j]^k$ the weight of a shortest path from i to j whose intermediate vertices all lie in $\{1, 2, \dots, k\}$, the recurrence is $d[i][j]^0 = w(i, j)$ and $d[i][j]^k = \min(d[i][j]^{k-1}, d[i][k]^{k-1} + d[k][j]^{k-1})$. The matrix D^n holds the final all pairs shortest path distances, and a negative diagonal entry would signal a negative weight cycle [C0047].

****Setup.**** The tool requires numbers, so each ``null`` (absent edge) is replaced by a very large number, 99999, standing for ∞ . The input matrix is:

```
| D^(0) | 1 | 2 | 3 | 4 |
|---|---|---|---|
| **1** | 0 | 3 | 99999 | 7 |
| **2** | 8 | 0 | 2 | 99999 |
| **3** | 5 | 99999 | 0 | 1 |
| **4** | 2 | 99999 | 99999 | 0 |
```

****Stage $k = 1$ **** (intermediate vertices from $\{1..1\}$) – updates: $d[2][4]: 99999 \rightarrow 15$ via 1; $d[3][2]: 99999 \rightarrow 8$ via 1; $d[4][2]: 99999 \rightarrow 5$ via 1.

```
| D^(1) | 1 | 2 | 3 | 4 |
|---|---|---|---|
| **1** | 0 | 3 | 99999 | 7 |
| **2** | 8 | 0 | 2 | 15 |
| **3** | 5 | 8 | 0 | 1 |
| **4** | 2 | 5 | 99999 | 0 |
```

****Stage $k = 2$ **** (intermediate vertices from $\{1..2\}$) – updates: $d[1][3]: 99999 \rightarrow 5$ via 2; $d[4][3]: 99999 \rightarrow 7$ via 2.

```
| D^(2) | 1 | 2 | 3 | 4 |
|---|---|---|---|
| **1** | 0 | 3 | 5 | 7 |
| **2** | 8 | 0 | 2 | 15 |
| **3** | 5 | 8 | 0 | 1 |
| **4** | 2 | 5 | 7 | 0 |
```

****Stage $k = 3$ **** (intermediate vertices from $\{1..3\}$) – updates: $d[1][4]: 7 \rightarrow 6$ via 3; $d[2][1]: 8 \rightarrow 7$ via 3; $d[2][4]: 15 \rightarrow 3$ via 3.

```
| D^(3) | 1 | 2 | 3 | 4 |
|---|---|---|---|
| **1** | 0 | 3 | 5 | 6 |
| **2** | 7 | 0 | 2 | 3 |
| **3** | 5 | 8 | 0 | 1 |
| **4** | 2 | 5 | 7 | 0 |
```

****Stage $k = 4$ **** (intermediate vertices from $\{1..4\}$) – updates: $d[2][1]: 7 \rightarrow 5$ via 4; $d[3][1]: 5 \rightarrow 3$ via 4; $d[3][2]: 8 \rightarrow 6$ via 4.

```
| D^(4) | 1 | 2 | 3 | 4 |
|---|---|---|---|
| **1** | 0 | 3 | 5 | 6 |
| **2** | 5 | 0 | 2 | 3 |
| **3** | 3 | 6 | 0 | 1 |
| **4** | 2 | 5 | 7 | 0 |
```

****Final all pairs shortest path matrix D^4 ****

[0356][5023][3601][2570]

No diagonal entry became negative, so the graph has no negative weight cycle [C0047]. The algorithm is three nested loops over k, i and j, giving running time **$\Theta(V^3)$** and space **$\Theta(V^2)$** [C0047]; compared with running Bellman-Ford from every source at $O(V^2 E)$, Floyd-Warshall is preferable on dense graphs [C0048].

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer faithfully reproduces the verified solver's stage-by-stage matrices and final result [[0,3,5,6],[5,0,2,3],[3,6,0,1],[2,5,7,0]], correctly handling nulls as infinity and matching all context facts.

Execution Latency: 38.6s

Q07: [NUMERIC] — PYQ 2024-25 SECTION B 2(c)

Query Prompt: Trace Dijkstra's algorithm from source A on the graph {"A": {"B": 4, "C": 2}, "B": {"C": 5, "D": 10}, "C": {"E": 3}, "D": {"F": 11}, "E": {"D": 4}, "F": {}} and give the final shortest distances.

Retrieval Telemetry: Topic dijkstra (Unit 3), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(1 on-topic), Max Similarity: 0.712

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0034	dijkstra	3	0.03128	0.7103	27.461	4.5 Single Source Shortest Path > Dijkstra's Alg
C0026	huffman_coding	3	0.03202	0.6516	45.602	3.4 Huffman Coding
C0035	bellman_ford	3	0.03055	0.6134	30.246	4.5 Single Source Shortest Path > Bellman-Ford A

Solvers Executed: dijkstra

Diagrams Offered: DIAG_DAA_U3_DIJKSTRA_01 | **Emitted:** DIAG_DAA_U3_DIJKSTRA_01

Raw Generated Output:

****Principle.**** Dijkstra's algorithm solves the single-source shortest path problem for graphs with **non-negative edge weights** using a greedy strategy: it maintains a priority queue keyed by shortest-path estimate and repeatedly extracts the vertex with minimum estimate, then relaxes all its outgoing edges (relaxation rule: if $d[u] + w(u,v) < d[v]$, update $d[v] = d[u] + w(u,v)$ and set predecessor $pi[v] = u$) [C0034].

****Trace from source A**** (distances shown after each extraction):

Iter	Extracted	d[extracted]	Relaxations performed	Distance array (A, B, C, D, E, F)	Visited set S
1	A	0	A-B: $\infty \rightarrow 4$; A-C: $\infty \rightarrow 2$	[0, 4, 2, ∞ , ∞ , ∞]	{A}
2	C	2	C-E: $\infty \rightarrow 5$	[0, 4, 2, ∞ , 5, ∞]	{A, C}
3	B	4	B-D: $\infty \rightarrow 14$	[0, 4, 2, 14, 5, ∞]	{A, C, B}
4	E	5	E-D: $14 \rightarrow 9$	[0, 4, 2, 9, 5, ∞]	{A, C, B, E}
5	D	9	D-F: $\infty \rightarrow 20$	[0, 4, 2, 9, 5, 20]	{A, C, B, E, D}
6	F	20	– (no outgoing edges)	[0, 4, 2, 9, 5, 20]	{A, C, B, E, D, F}

Key steps:

- A is extracted first with $d = 0$; edges A-B (4) and A-C (2) set initial estimates.
- C ($d = 2$) is extracted before B ($d = 4$) because it has the smaller key; relaxing C-E gives $d[E] = 2 + 3 = 5$.
- B's relaxation B-D would give $d[D] = 4 + 10 = 14$, but when E is extracted, E-D improves it to $d[D] = 5 + 4 = 9$ – exactly the kind of improvement the greedy extraction plus relaxation captures [C0034].
- D's relaxation sets $d[F] = 9 + 11 = 20$, and F is extracted last with no edges to relax.

****Shortest path tree (predecessors):**** B-A, C-A, E-C, D-E, F-D, i.e. shortest paths:
- A-B (4), A-C (2), A-C-E (5), A-C-E-D (9), A-C-E-D-F (20).

****Final shortest distances from A:**** $d[A] = 0$, $d[B] = 4$, $d[C] = 2$, $d[D] = 9$, $d[E] = 5$, $d[F] = 20$.

This matches the distance-array evolution shown in the Dijkstra shortest-path-tree trace [Diagram: DIAG_DAA_U3_DIJKSTRA_01].

****Complexity:**** $O((V + E) \log V)$ with a binary-heap priority queue [C0034].

Metric Scores: context_relevance=33.3% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer faithfully reproduces the verified solver trace and final distances (A:0, B:4, C:2, D:9, E:5, F:20) with correct relaxation steps, predecessors, and complexity, all grounded in the context.

Execution Latency: 27.73s

Q08: [NUMERIC] — authored

Query Prompt: Solve the fractional knapsack instance with weights [10, 20, 30], values [60, 100, 120] and capacity 50 using the greedy method. Show the value to weight ratios and the total profit.

Retrieval Telemetry: Topic `fractional_knapsack` (Unit 3), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.8345

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0023	fractional_knapsack	3	0.03279	0.8345	35.77	3.3 Fractional Knapsack Problem
C0024	fractional_knapsack	3	0.03062	0.7858	18.259	3.3 Fractional Knapsack Problem
C0066	job_sequencing	3	0.02908	0.5626	22.61	3.3b Job Sequencing with Deadlines

Solvers Executed: `fractional_knapsack`

Raw Generated Output:

```
# Fractional Knapsack – Greedy Solution

**Problem setup [C0023]:** n = 3 items with weights w = [10, 20, 30], values v = [60, 100, 120], knapsack
capacity W = 50. Since this is the fractional knapsack problem, we are allowed to take fractions of items,
so the greedy strategy applies: compute value-to-weight ratios  $v_i / w_i$ , sort in descending order, then
fill the knapsack, taking a fraction of the item that does not fit.

## Step 1 – Value-to-weight ratios

| Item | Weight  $w_i$  | Value  $v_i$  | Ratio  $v_i / w_i$  |
|-----|-----|-----|-----|
| 1 | 10 | 60 | **6.0** |
| 2 | 20 | 100 | **5.0** |
| 3 | 30 | 120 | **4.0** |

Sorted descending by ratio: Item 1 → Item 2 → Item 3.

## Step 2 – Greedy fill (tool output reproduced exactly)

| Item | Weight | Value | Ratio | Fraction taken | Value taken | Remaining capacity |
|-----|-----|-----|-----|-----|-----|-----|
| 1 | 10 | 60 | 6.0 | 1.0 | 60.0 | 40.0 |
| 2 | 20 | 100 | 5.0 | 1.0 | 100.0 | 20.0 |
| 3 | 30 | 120 | 4.0 | 0.6667 | 80.0 | 0.0 |

- **Item 1:**  $w = 10 \leq 50$ , so the whole item is taken: total_value = 60, remaining capacity = 40 [C0024,
lines 5-7].
- **Item 2:**  $w = 20 \leq 40$ , whole item taken: total_value = 160, remaining capacity = 20.
- **Item 3:**  $w = 30 > 20$ , so only the fraction  $20/30 = 0.6667$  is taken: total_value +=  $120 \times (20/30) = 80$ ,
then break [C0024, lines 8-10].

## Result

**Total profit =  $60 + 100 + 80 = 240.0$ ** – this is the optimal value for the instance.

**Complexity:** Computing the ratios is  $O(n)$ , sorting dominates at  $O(n \log n)$ , and the fill loop is  $O(n)$ ,
giving an overall time complexity of  $O(n \log n)$  [C0024].
```

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer correctly shows ratios 6, 5, 4, the greedy fill taking items 1 and 2 fully and 2/3 of item 3, total profit 240, and $O(n \log n)$ complexity, all fully grounded in the context and verified solver output.

Execution Latency: 39.33s

Q09: [DIAGRAM] — authored

Query Prompt: Draw the recursion tree for $T(n) = 2T(n/2) + n$ and explain why the total cost is $n \log n$.

Retrieval Telemetry: Topic `recurrences` (Unit 1), Confidence: 0.95, Router: llm+lexical, Filter: priority:topic(4 on-topic), Max Similarity: 0.6914

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0006	recurrences	1	0.03252	0.6914	25.609	1.2 Recurrences & Master Theorem > The Master Th
C0005	recurrences	1	0.03252	0.6499	26.281	1.2 Recurrences & Master Theorem > The Master Th
C0007	recurrences	1	0.03175	0.627	23.11	1.2 Recurrences & Master Theorem > Example of Ma
C0004	recurrences	1	0.01471	0.5638	5.741	1.2 Recurrences & Master Theorem

Solvers Executed: `recursion_tree`

Diagrams Offered: DIAG_DAA_U1_RECUR_TREE_01 | **Emitted:** DIAG_DAA_U1_RECUR_TREE_01

Raw Generated Output:

```
# Recursion Tree for  $T(n) = 2T(n/2) + n$ 

**Principle.** A recurrence is an equation describing a function in terms of its value on smaller inputs,
and it is the standard tool for analyzing divide-and-conquer running times [C0004]. The recursion-tree
method makes the Master Theorem form  $T(n) = aT(n/b) + f(n)$  concrete [C0005]: the root holds the size- $n$ 
problem, each node spawns a children of size  $n/b$ , and each node pays  $f(\text{size})$  for its local work.

## Step 1 – Identify parameters

For  $T(n) = 2T(n/2) + n$ :  $a = 2$ ,  $b = 2$ ,  $f(n) = n$  [C0007].

## Step 2 – Expand the tree

Expanding with  $a = 2$ ,  $b = 2$ ,  $f_{\text{exponent}} = 1$  gives exactly this structure (tool output reproduced verbatim):

| Level | Nodes | Subproblem size | Cost per node | Level cost | Level-cost coefficient |
|-----|-----|-----|-----|-----|-----|
| 0 | 1 |  $n$  |  $(n)^{a_1}$  |  $(2/2^{a_1})^{a_0} \cdot n^{a_1}$  | 1 |
| 1 |  $2^{a_1}$  |  $n/2^{a_1}$  |  $(n/2^{a_1})^{a_1}$  |  $(2/2^{a_1})^{a_1} \cdot n^{a_1}$  | 1 |
| 2 |  $2^{a_2}$  |  $n/2^{a_2}$  |  $(n/2^{a_2})^{a_1}$  |  $(2/2^{a_1})^{a_2} \cdot n^{a_1}$  | 1 |
| 3 |  $2^{a_3}$  |  $n/2^{a_3}$  |  $(n/2^{a_3})^{a_1}$  |  $(2/2^{a_1})^{a_3} \cdot n^{a_1}$  | 1 |

- **Height** of the tree:  $\log_2(n)$  – the problem size halves at each level until it reaches a constant.
- **Leaf count:**  $n^{a_1}(\log_2 2) = n^{a_1} \cdot 1 = n$  leaves.
- **Verdict (tool):** "Every level costs the same, so the total is the level cost times the height."
- **Total:**  $\Theta(n \log n)$ .

[Diagram: DIAG_DAA_U1_RECUR_TREE_01] shows this same expansion:  $\log n$  levels, each carrying total cost  $n$ .

## Step 3 – Why the total is  $n \log n$ 

1. **Per-level cost is constant.** At level  $i$  there are  $2^i$  nodes, each of size  $n/2^i$ , so level  $i$  costs  $2^i \cdot (n/2^i) = n$ . Doubling the number of nodes exactly cancels the halving of the subproblem size – the coefficient at every level is 1, as the table shows.
2. **The height is  $\log_2 n$ .n at each level takes  $\log_2 n$  levels to reach the leaves [C0005].
3. **Sum over all levels.** Total =  $n + n + \dots + n$  ( $\log_2 n$  terms) plus  $\Theta(n)$  for the  $n$  leaves =  $n \cdot \log_2 n$ . This is the "cost evenly distributed across the levels" situation, i.e. Case 2 of the Master Theorem: if  $f(n) = \Theta(n^{\log_b a})$  then  $T(n) = \Theta(n^{\log_b a} \log n)$  [C0006].

## Step 4 – Cross-check with the Master Theorem

This is precisely the Merge Sort recurrence [C0007]:  $a = 2$ ,  $b = 2$  gives  $n^{a_1}(\log_2 2) = n$ ; since  $f(n) = \Theta(n^{\log_b a})$ , Case 2 applies [C0006], confirming the tree's verdict.

- **Final result:**  $T(n) = \Theta(n \log n)$ .
```

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer fully explains the recursion tree for $T(n)=2T(n/2)+n$, grounding every claim in the retrieved context and solver output, with correct level costs (n each), height $\log_2 n$, leaf count n , and total $\Theta(n \log n)$.

Execution Latency: 28.64s

Q10: [DIAGRAM] — authored

Query Prompt: Show the rotation that repairs an LL imbalance in an AVL tree and state the balance factor before and after.

Retrieval Telemetry: Topic red_black_trees (Unit 2), Confidence: 0.712, Router: lexical, Filter: priority:topic(5 on-topic), Max Similarity: 0.7548

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0059	red_black_trees	2	0.03279	0.7548	38.913	2.5 AVL Trees and Rotations
C0058	red_black_trees	2	0.03226	0.7336	27.377	2.5 AVL Trees and Rotations
C0060	red_black_trees	2	0.03175	0.7121	26.992	2.5 AVL Trees and Rotations
C0013	red_black_trees	2	0.02724	0.4218	8.509	2.1 Red-Black Trees
C0014	red_black_trees	2	0.01562	0.4814	7.161	2.1 Red-Black Trees > Properties of Red-Black Tr

Diagrams Offered: DIAG_DAA_U2_AVL_ROT_01 | **Emitted:** DIAG_DAA_U2_AVL_ROT_01

Raw Generated Output:

```
## Principle

An AVL tree is a BST in which every node's balance factor – height of left subtree minus height of right subtree – must lie in {-1, 0, +1}. When the offending node lies in the **left subtree of the left child** of the unbalanced node (the **LL case**), exactly **one single right rotation** repairs it [C0058].

## Step 1 – Build the imbalance

Insert 30, then 20, then 10 into an empty AVL tree [C0059]:
```

Before rotation: Balance factors 30 30 : +2 (left subtree height 2, right subtree height 0) / 20 20 : +1 / 10

```
After inserting 10, the left subtree of 30 has height 2 and its right subtree has height 0, so the balance factor of 30 becomes **+2** and the AVL property is violated at 30. Since 10 sits in the left subtree of the left child 20, this is the LL case [C0059].

## Step 2 – Single right rotation about 30

Node 20 rises to the root, taking 10 as its left child and 30 as its right child [C0059]. The rotation is shown in [Diagram: DIAG_DAA_U2_AVL_ROT_01].
```

After rotation: Balance factors 20 20 : 0 / \ 10 30 10 : 0, 30 : 0

```
## Balance factors before and after

| Node | Before rotation | After rotation |
|-----|-----|-----|
| 30    | **+2** (violates AVL) | 0 |
| 20    | +1 | 0 |
| 10    | 0 | 0 |

The rotation restores every balance factor to 0, and the tree height drops from 3 to 2 [C0059].

## Complexity

A rotation runs in **O(1)** time, and since the height of an AVL tree with n nodes is O(log n), search, insert and delete are all **O(log n)** [C0060].
```

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer faithfully uses the worked LL example from the context, correctly shows the single right rotation, states balance factors before (+2 at 30, +1 at 20) and after (all 0), and gives correct O(1) rotation and O(log n) operation complexities.

Execution Latency: 29.93s

Q11: [DISAMBIGUATION] — authored

Query Prompt: Explain how edge relaxation works in Dijkstra's algorithm and why it requires non-negative edge weights.

Retrieval Telemetry: Topic dijkstra (Unit 3), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(1 on-topic), Max Similarity: 0.7441

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0034	dijkstra	3	0.03279	0.7441	22.578	4.5 Single Source Shortest Path > Dijkstra's Alg
C0035	bellman_ford	3	0.03226	0.6749	16.241	4.5 Single Source Shortest Path > Bellman-Ford A
C0033	mst	3	0.03080	0.6163	13.464	4.4 Graph Algorithms: Traversals & MST > Minimum

Raw Generated Output:

```
# Edge Relaxation in Dijkstra's Algorithm

## 1. The setting

Dijkstra's algorithm solves the single-source shortest path problem on graphs with non-negative edge weights. It works greedily: it keeps a priority queue of vertices keyed by their current shortest-path estimate and repeatedly extracts the vertex with the minimum estimate [C0034]. Every vertex `v` therefore carries a distance estimate `v.d`, and `Initialize-Single-Source(G, s)` is called first to set these estimates up before the queue is built [C0034].

## 2. What relaxation is

Relaxation is the operation that *improves* a distance estimate whenever a shorter route to a vertex is discovered. In the pseudocode it appears as the call `Relax(u, v, w)` made for every neighbour `v` of the extracted vertex `u` [C0034]. The test it applies is exactly the comparison shown in the notes' negative-cycle check:

> `if v.d > u.d + w(u, v):` — [C0035]

i.e. relaxation asks: *can we reach `v` more cheaply by going through `u`?*

- **If yes** (`v.d > u.d + w(u, v)`): the path through `u` is shorter, so `v.d` is lowered to `u.d + w(u, v)` and `u` is recorded as `v`'s predecessor on the shortest path. Since the queue is keyed by distance [C0034], the queue ordering is correspondingly updated.
- **If no** (`v.d ≤ u.d + w(u, v)`): the current estimate for `v` is already at least as good, so nothing changes.

So in Dijkstra's loop: extract the minimum vertex `u`, add it to the settled set `S`, and then relax all edges out of `u` [C0034].

(Note: the notes give `Initialize-Single-Source` and `Relax` by name only; the comparison condition is the part they make explicit [C0035]. The standard convention the name implies — source distance 0, all others ∞ — is assumed here.)

## 3. Why non-negative edge weights are required

Dijkstra's correctness depends on a greedy invariant: **once a vertex is extracted from the queue and added to `S`, its distance is final.** This is safe only if extending a path can never decrease its cost. With non-negative weights, any relaxation performed later can only be triggered by edges of weight ≥ 0, so no edge discovered later can produce a shorter route to an already-settled vertex.

If a negative-weight edge exists, a path discovered *after* `u` is settled could still reach `u` (or an already-extracted vertex) with a smaller total cost, so the invariant breaks: the algorithm would have committed to a non-minimal distance and never revisits that vertex. This is precisely why the notes state plainly: **"If negative weight edges exist, Dijkstra's algorithm may fail" [C0034]. The notes state this limitation as a fact; they do not include a worked counterexample.

This is exactly the gap Bellman-Ford fills: it solves the single-source problem even with negative edge weights, and detects reachable negative cycles, by relaxing **all** edges `|V| - 1` times — which works because, absent negative cycles, a shortest path has at most `|V| - 1` edges [C0035].

## 4. Complexity

Dijkstra runs in  $O((V + E) \log V)$  with a binary heap [C0034]; Bellman-Ford, the alternative that tolerates negative weights, runs in  $O(VE)$  [C0035].

**Final answer:** Relaxation is the improvement step `if v.d > u.d + w(u, v)` — lower `v.d` through `u` and record the predecessor — applied to all neighbours of each extracted minimum vertex [C0034] [C0035]. Non-negative weights are required because Dijkstra's greedy choice is only final when no later edge can cheapen a settled vertex's path; negative edges violate this, so Dijkstra may fail and Bellman-Ford must be used instead [C0034] [C0035].
```


Metric Scores: context_relevance=33.3% | context_precision=100.0% | context_recall=70.0% | faithfulness=80.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer correctly explains relaxation and the non-negative weight requirement with sound reasoning, but the context lacks the explicit Relax/Initialize-Single-Source definitions (the answer fills this gap with standard knowledge), so recall and faithfulness are slightly below full marks.

Execution Latency: 59.63s

Q12: [DISAMBIGUATION] — authored

Query Prompt: Explain edge relaxation in the Bellman-Ford algorithm and why V-1 passes over all edges are needed.

Retrieval Telemetry: Topic `bellman_ford` (Unit 3), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(1 on-topic), Max Similarity: 0.706

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0035	bellman_ford	3	0.03279	0.706	22.51	4.5 Single Source Shortest Path > Bellman-Ford A
C0033	mst	3	0.02921	0.548	13.228	4.4 Graph Algorithms: Traversals & MST > Minimum
C0034	dijkstra	3	0.02838	0.562	9.37	4.5 Single Source Shortest Path > Dijkstra's Alg

Solvers Executed: `bellman_ford`

Raw Generated Output:

```

**Trace output (reproduced exactly from the tool):** graph `s-a (1)`, `a-b (1)`, `b-c (1)`, source `s`; |V|
= 4, so **|V| - 1 = 3 passes** over the 3 edges; `null` denotes  $\infty$  from `Initialize-Single-Source` [C0035].

| Pass | Relaxations that improved an estimate (old  $\rightarrow$  new) | Distances after the pass (s, a, b, c) |
|---|---|---|
| 1 | `s-a`: null  $\rightarrow$  1.0 | s = 0.0, a = 1.0, b = null, c = null |
| 2 | `a-b`: null  $\rightarrow$  2.0; `b-c`: null  $\rightarrow$  3.0 | s = 0.0, a = 1.0, b = 2.0, c = 3.0 |
| 3 | (none - all edges pass the check `v.d > u.d + w(u,v)` and fail it) | s = 0.0, a = 1.0, b = 2.0, c = 3.0 |

Tool output: `negative_cycle: false`, final distances `{s: 0.0, a: 1.0, b: 2.0, c: 3.0}`, complexity
`O(VE)`.

**What the trace shows:** with the edges given in the unfavourable order, pass 1 settles only the first edge
(a = 1), pass 2 settles the second and third (b = 2, c = 3), and pass 3 changes nothing. In the worst
ordering, one pass can advance a correct distance along just one edge of the shortest path - which is
exactly why the loop must run `for i = 1 to |V| - 1` [C0035]. Since every relaxation failed on the final
checking pass, no negative cycle exists, matching the tool's `negative_cycle: false` and the return TRUE
condition of the pseudocode [C0035].

## Final answer

- **Relaxation** = the test `v.d > u.d + w(u, v)` followed by the update `v.d = u.d + w(u, v)` (with
predecessor update); it is the core of both Bellman-Ford and its negative-cycle check [C0035].
- **|V| - 1 passes** are needed because, absent negative cycles, every shortest path has at most |V| - 1
edges, and one full pass of relaxations can settle at least (worst case, exactly) one more edge of each
shortest path [C0035].
- **Time Complexity: O(VE)** [C0035] - in general,  $O((V + E) \log V)$  is Dijkstra's bound for the non-negative
case [C0034]; here with |V| = 4 and E = 3, cost is  $O(4 \cdot 3) = O(12)$  edge relaxations, as the trace confirms.

```

Metric Scores: context_relevance=33.3% | context_precision=100.0% | context_recall=60.0% | faithfulness=40.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context gives the pseudocode, the |V|-1-edge shortest-path justification, and the verified trace, but never defines the relaxation test/update, so the answer's core definition of relaxation is unsupported textbook knowledge, capping faithfulness below 0.5.

Execution Latency: 134.96s

Q13: [DISAMBIGUATION] — authored

Query Prompt: How is the knapsack problem solved by the greedy method when an item may be broken into fractions?

Retrieval Telemetry: Topic `fractional_knapsack` (Unit 3), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.8389

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0023	fractional_knapsack	3	0.03226	0.8252	19.759	3.3 Fractional Knapsack Problem
C0024	fractional_knapsack	3	0.02853	0.7524	7.95	3.3 Fractional Knapsack Problem
C0020	divide_and_conquer	3	0.03033	0.6218	13.059	3.1 Divide and Conquer Paradigm

Raw Generated Output:

```
## Fractional Knapsack Problem – Greedy Solution
```

```
**Problem statement:** In the Fractional Knapsack problem, we are given n items, each with a weight w_i and a value v_i, and a knapsack of capacity W. The goal is to maximize the total value in the knapsack. Unlike the 0/1 knapsack problem, we are allowed to take fractions of items [C0023].
```

```
**Greedy strategy:** The greedy choice is based on the value-to-weight ratio of each item, since fractions are permitted and the best "value density" should fill the knapsack first [C0023]:
```

1. Calculate the value-to-weight ratio v_i / w_i for each item.
2. Sort the items in descending order of this ratio.
3. Iterate through the sorted items: if the knapsack has enough capacity, take the whole item. If not, take the fractional amount needed to fill the knapsack [C0023].

```
**Algorithm** [C0024]:
```

Algorithm Fractional-Knapsack(v, w, W) 1. Compute $ratio[i] = v[i] / w[i]$ for $i = 1$ to n 2. Sort items descending by ratio 3. $total_value = 0$ 4. for $i = 1$ to n : 5. if $w[i] \leq W$: 6. $total_value = total_value + v[i]$ 7. $W = W - w[i]$ 8. else: 9. $total_value = total_value + v[i] * (W / w[i])$ 10. break 11. return $total_value$

```
**Working of the algorithm:**
```

- Step 1-2 establish the greedy ordering: highest ratio (most valuable per unit weight) items are considered first.
- Step 4-7 keep placing whole items while capacity W remains sufficient, reducing W by each item's weight.
- At the first item that does not fit (step 8), only the fraction $v[i] \times (W / w[i])$ is added – exactly enough to fill the remaining capacity – and the loop terminates with `break` (step 10), since the knapsack is then full [C0024].

```
**Complexity:** The sorting step takes  $O(n \log n)$ , and the iteration takes  $O(n)$ . The overall time complexity is  $O(n \log n)$  [C0024]. (Since sorting dominates, this is also the tight bound, i.e.,  $\Theta(n \log n)$ .)
```

```
**Final result:** The greedy method – sort by  $v_i/w_i$  descending, take items whole while they fit, then take a fraction of the first item that does not fit – returns the maximum total value in  $O(n \log n)$  time [C0023] [C0024].
```

```
*Note: No specific numerical instance (weights, values, capacity) was supplied in the question, so no numerical trace is reproduced; the algorithm above fully specifies the solution method.*
```

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer faithfully restates the context's complete greedy strategy, algorithm, working, and $O(n \log n)$ complexity with no unsupported additions.

Execution Latency: 26.85s

Q14: [DISAMBIGUATION] — authored

Query Prompt: How is the knapsack problem solved when items cannot be broken and each item is either taken whole or left?

Retrieval Telemetry: Topic `knapsack_01` (Unit 4), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.7698

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0030	knapsack_01	4	0.03279	0.7698	27.298	4.3 0/1 Knapsack Problem
C0031	knapsack_01	4	0.03126	0.6583	15.131	4.3 0/1 Knapsack Problem
C0027	dp_principles	4	0.02817	0.5715	10.378	4.1 Dynamic Programming Principles

Raw Generated Output:

```
## 0/1 Knapsack Problem – Solved by Dynamic Programming

**Problem statement.** When items cannot be broken into fractions, the problem becomes the **0/1 Knapsack problem**: for each item you either take it (1) or leave it (0) [C0030].

**Why greedy fails here.** Because items are indivisible, the greedy strategy does not work properly – taking a high-value item might prevent utilizing the remaining capacity effectively [C0030]. Hence DP is used instead.

**Why DP applies.** The problem satisfies the two properties DP requires [C0027]:
1. **Optimal substructure** – the optimal solution is built from optimal solutions of subproblems (smaller item counts, smaller capacities).
2. **Overlapping subproblems** – the same subproblems recur, so each is solved once and its answer saved in a table, avoiding recomputation [C0027].

**DP formulation.** Let DP[i][w] be the maximum value obtainable using the first i items with maximum weight capacity w [C0030].

**Algorithm** (01-Knapsack(v, wt, W, n)) [C0030]:
1. Create a 2D array DP[n+1][W+1] initialized to 0.
2. For i = 1 to n:
3.     For w = 1 to W:
4.         If wt[i] <= w:
5-6.         DP[i][w] = max(v[i] + DP[i-1][w - wt[i]], DP[i-1][w]) – the maximum of including the item or excluding it.
7.         Else:
8-9.         DP[i][w] = DP[i-1][w] – the item cannot be included.
10. Return DP[n][W].

**Recurrence (the decision at each cell):**

$$
DP[i][w] = \begin{cases} \max(v[i] + DP[i-1][w - wt[i]], DP[i-1][w]) & \text{if } wt[i] \leq w \\ DP[i-1][w] & \text{otherwise} \end{cases}
$$

That is, at every cell the algorithm compares the value of taking item i (its value plus the best answer for the leftover capacity w - wt[i]) against the value of skipping it, and stores the larger [C0030].

**Result.** The answer to the original problem is the final table entry DP[n][W] [C0030].

**Complexity.** The time and space complexity is  $O(nW)$ , where n is the number of items and W is the knapsack capacity. This is pseudo-polynomial time, because it depends on the magnitude of W rather than only on the input size [C0031].
```

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context fully covers the 0/1 knapsack DP formulation, algorithm, and complexity, and the answer faithfully restates it with correct recurrence and $O(nW)$ pseudo-polynomial complexity.

Execution Latency: 11.05s

Q15: [DISAMBIGUATION] — authored

Query Prompt: Which dynamic programming algorithm finds the best order to multiply a chain of matrices, and what does its table store?

Retrieval Telemetry: Topic `matrix_chain` (Unit 4), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(1 on-topic), Max Similarity: 0.6798

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0029	matrix_chain	4	0.03202	0.6798	13.041	4.2 Matrix Chain Multiplication
C0027	dp_principles	4	0.03227	0.6545	18.207	4.1 Dynamic Programming Principles
C0028	dp_principles	4	0.03226	0.6567	16.586	4.1 Dynamic Programming Principles

Raw Generated Output:

```
Algorithm: The dynamic programming algorithm for this problem is Matrix Chain Multiplication [C0029]. Given a chain of n matrices A1, A2, ..., An, it exploits the associativity of matrix
```

multiplication to find a full parenthesization of the product that minimizes the number of scalar multiplications, where multiplying a $p \times q$ matrix by a $q \times r$ matrix costs $p \times q \times r$ scalar multiplications [C0029].

What the table stores: The table entry $m[i, j]$ stores the minimum number of scalar multiplications needed to compute the matrix product $A_{\{i..j\}}$ [C0029]. It is filled using the recurrence, with the dimensions of matrix A_i being $P_{\{i-1\}} \times P_i$ [C0029]:

```
- m[i, j] = 0 if i = j
- m[i, j] = min over i ≤ k < j of { m[i, k] + m[k+1, j] + P_{i-1} P_k P_j } if i < j
```

Why DP applies: The problem exhibits the two properties DP requires – optimal substructure (the optimal parenthesization of $A_{\{i..j\}}$ splits at some k into optimal parenthesizations of $A_{\{i..k\}}$ and $A_{\{k+1..j\}}$) and overlapping subproblems (the same subchains are needed by many larger subproblems, so each is solved once and saved in the table) [C0027]. The bottom-up (tabulation) style of filling the table is one of the two standard DP implementation approaches [C0028].

Complexity: The table is $n \times n$ and each entry takes $O(n)$ time to compute, giving a time complexity of $O(n^3)$ and a space complexity of $O(n^2)$ [C0029].

Metric Scores: context_relevance=33.3% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context fully covers Matrix Chain Multiplication, the $m[i,j]$ table, recurrence, and complexities, and the answer faithfully and correctly restates them.

Execution Latency: 6.47s

Q16: [DISAMBIGUATION] — authored

Query Prompt: Which dynamic programming algorithm computes shortest paths between every pair of vertices, and what does its table store?

Retrieval Telemetry: Topic floyd_warshall (Unit 4), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.6746

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0047	floyd_warshall	4	0.03202	0.6504	17.662	4.4b All Pairs Shortest Paths: Floyd-Warshall
C0048	floyd_warshall	4	0.03128	0.6267	14.555	4.4b All Pairs Shortest Paths: Floyd-Warshall
C0069	branch_and_bound	4	0.03200	0.6634	13.72	KCS-503 detailed syllabus

Raw Generated Output:

Algorithm: Floyd-Warshall – a dynamic programming algorithm that computes shortest path distances between every pair of vertices of a weighted directed graph. Negative edge weights are allowed, but negative weight cycles are not. [C0047]

What the table stores: The table entry $d[i][j]^{(k)}$ stores the weight of a shortest path from vertex i to vertex j whose intermediate vertices all lie in the set $\{1, 2, \dots, k\}$. It is built up by the recurrence: [C0047]

```
-  $d[i][j]^{(0)} = w(i, j)$  (no intermediate vertices allowed, so just the direct edge weight)
-  $d[i][j]^{(k)} = \min( d[i][j]^{(k-1)}, d[i][k]^{(k-1)} + d[k][j]^{(k-1)} )$ 
```

i.e., at stage k the algorithm decides whether allowing vertex k as an intermediate vertex improves the path from i to j . After the loops finish, the matrix $D^{(n)}$ holds the final all pairs shortest path distances; a diagonal entry that becomes negative signals a negative weight cycle. [C0047]

Algorithm (as given): [C0047]

FLOYD-WARSHALL(W) 1. $n = \text{rows}[W]$; $D = W$ 2. for $k = 1$ to n 3. for $i = 1$ to n 4. for $j = 1$ to n 5. $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$ 6. return D

Complexity: The three nested loops over k , i and j give running time $\Theta(V^3)$ and space $\Theta(V^2)$. [C0047]

Remark: Warshall's algorithm is the same triple loop with min/plus replaced by OR/AND and computes the transitive closure in $\Theta(V^3)$. Compared with running Bellman-Ford from every source, which costs $O(V^2 E)$, Floyd-Warshall is preferable on dense graphs. [C0048]

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer is fully grounded in context C0047/C0048, directly identifies Floyd-Warshall and explains what the table stores, with correct recurrence, pseudocode, and complexity.

Execution Latency: 16.59s

Q17: [DISAMBIGUATION] — PYQ 2022-23 SECTION B 2(c)

Query Prompt: Which greedy algorithm builds a minimum spanning tree by repeatedly adding the lightest edge that creates no cycle?

Retrieval Telemetry: Topic `mst` (Unit 3), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(1 on-topic), Max Similarity: 0.7721

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0033	mst	3	0.03279	0.7721	20.023	4.4 Graph Algorithms: Traversals & MST > Minimum
C0034	dijkstra	3	0.03226	0.6592	10.913	4.5 Single Source Shortest Path > Dijkstra's Alg
C0021	greedy_general	3	0.03126	0.6209	10.654	3.2 Greedy Algorithms Overview

Raw Generated Output:

```

**Answer: Kruskal's Algorithm** [C0033]

**Principle.** Kruskal's algorithm is a greedy algorithm for finding a Minimum Spanning Tree (MST) of a connected, undirected, weighted graph – a subgraph that is a tree, includes all vertices, and has the minimum possible total edge weight [C0033]. It follows the greedy strategy of making the locally optimal choice (the smallest available edge) at each step [C0021].

**Method.**
1. Sort all edges in non-decreasing order of their weight.
2. Pick the smallest edge.
3. If it does not form a cycle with the already included edges (checked using a Disjoint Set Union data structure), add it to the MST.
4. Repeat until the spanning tree is complete [C0033].

The cycle check is exactly the distinguishing feature asked about: an edge is rejected precisely when it would create a cycle [C0033].

**Complexity:**  $O(E \log E)$ , dominated by the edge sort [C0033].

**Contrast.** Prim's algorithm, the other greedy MST method, instead grows the tree from a starting vertex by always picking the minimum-weight edge connecting the "included" and "not yet included" vertex sets – it never faces a cycle-formation choice, since it only ever adds one edge out of the tree. Its complexity with a min-heap is  $O(E \log V)$  [C0033].

```

Metric Scores: context_relevance=33.3% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context fully describes Kruskal's algorithm (sort edges, add lightest non-cycle-forming edge, $O(E \log E)$), and the answer correctly identifies it with accurate, fully supported details and a correct Prim's contrast.

Execution Latency: 6.4s

Q18: [DISAMBIGUATION] — authored

Query Prompt: Which greedy algorithm builds an optimal prefix code by repeatedly merging the two lowest frequency symbols?

Retrieval Telemetry: Topic `huffman_coding` (Unit 3), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.7173

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0025	huffman_coding	3	0.03279	0.7173	14.653	3.4 Huffman Coding
C0026	huffman_coding	3	0.02964	0.7166	5.221	3.4 Huffman Coding

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0021	greedy_general	3	0.03175	0.6433	12.294	3.2 Greedy Algorithms Overview

Raw Generated Output:

****Answer: Huffman coding**** – the greedy algorithm that constructs an optimal prefix code for lossless data compression by repeatedly merging the two lowest-frequency symbols [C0025].

****Principle.**** Huffman coding assigns variable-length codes to input characters, with code lengths based on character frequencies – the most frequent character gets the shortest code. It is a greedy method: it makes the locally optimal choice (merge the two smallest frequencies) at each step, which leads to a globally optimal prefix-free code [C0025], [C0021].

****Algorithm.**** A priority queue (min-heap) keyed on frequency builds the tree bottom-up [C0025]:

Algorithm Huffman(C) // C is a set of n characters and their frequencies
 1. $n = |C|$
 2. $Q = C$ // Priority queue based on frequency
 3. for $i = 1$ to $n - 1$
 4. allocate a new node z
 5. $z.left = x = \text{Extract-Min}(Q)$
 6. $z.right = y = \text{Extract-Min}(Q)$
 7. $z.freq = x.freq + y.freq$
 8. $\text{Insert}(Q, z)$
 9. return $\text{Extract-Min}(Q)$ // Return the root of the tree

Each iteration extracts the two minimum-frequency nodes, merges them into a new node whose frequency is their sum, and reinserts it – exactly the "repeatedly merging the two lowest frequency symbols" behaviour the question describes [C0025].

****Trace (frequencies a:45, b:13, c:12, d:16, e:9, f:5)**** [C0026]:

Step	Action
1	Extract f(5) and e(9) → combine into node of 14
2	Extract c(12) and b(13) → combine into 25
3	Extract (fe):14 and d:16 → combine into 30
4	Extract (cb):25 and (fed):30 → combine into 55
5	Extract a:45 and (cbfed):55 → combine into 100 (Root)

Traversing left edges as '0' and right edges as '1' yields the prefix-free binary codes for optimal compression [C0026].

****Complexity.**** The loop runs $n - 1$ times, each iteration performing $O(\log n)$ heap operations, giving a total time complexity of $O(n \log n)$ [C0026].

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer correctly identifies Huffman coding and reproduces the algorithm, trace, and $O(n \log n)$ complexity exactly as grounded in the context.

Execution Latency: 6.76s

Q19: [GENERAL] — PYQ 2022-23 SECTION A 1(b)

Query Prompt: Explain and compare the best case and worst case time complexity of Quick Sort.

Retrieval Telemetry: Topic sorting_comparison (Unit 1), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(6 on-topic), Max Similarity: 0.755

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0011	sorting_comparison	1	0.03252	0.755	14.01	1.3 Advanced Sorting Algorithms > Quick Sort
C0010	sorting_comparison	1	0.03252	0.699	14.815	1.3 Advanced Sorting Algorithms > Quick Sort
C0009	sorting_comparison	1	0.03175	0.6481	13.652	1.3 Advanced Sorting Algorithms > Merge Sort
C0068	sorting_comparison	1	0.03008	0.6325	11.304	KCS-503 detailed syllabus
C0012	sorting_comparison	1	0.02988	0.6342	9.973	1.3 Advanced Sorting Algorithms > Heap Sort

Solvers Executed: master_theorem

Raw Generated Output:

Quick Sort: Best Case vs Worst Case Time Complexity

Principle

Quick Sort follows the divide-and-conquer paradigm, but unlike merge sort it does all the heavy lifting in the **partition phase**: it selects a **pivot** element and partitions the array so that elements smaller than the pivot lie to its left and larger elements to its right [C0010]. The running time is therefore governed by **how balanced the partitions are**, which depends on the pivot.

Algorithm [C0010]:

Algorithm QuickSort(A, p, r) 1. if $p < r$ 2. $q = \text{Partition}(A, p, r)$ 3. QuickSort(A, p, $q - 1$) 4. QuickSort(A, $q + 1$, r)

Algorithm Partition(A, p, r) 1. $x = A[r]$ // choosing last element as pivot 2. $i = p - 1$ 3. for $j = p$ to $r - 1$ 4. if $A[j] \leq x$ 5. $i = i + 1$ 6. exchange $A[i]$ with $A[j]$ 7. exchange $A[i + 1]$ with $A[r]$ 8. return $i + 1$

Best Case – $\Theta(n \log n)$

When it occurs: the partition process always divides the array into **two nearly equal halves** [C0010]. Each partition pass costs $\Theta(n)$, and it spawns two subproblems of size $n/2$:

$$T(n) = 2T(n/2) + \Theta(n)$$

Solving by the Master Theorem ($a = 2$, $b = 2$, $f(n) = n$):

Step	Result
Recurrence	$T(n) = 2T(n/2) + n$
Critical exponent	$\log_b(a) = \log_2(2) = 1$
Comparison	$f(n) = n$ matches $n^{\log_b a} = n^1$
Applicable case	Case 2
Solution	$T(n) = \Theta(n \log n)$

The notes state this leads to **$\Theta(n \log n)$** [C0010]; the Master Theorem sharpens it to the tight bound **$\Theta(n \log n)$** .

Worst Case – $\Theta(n^2)$

When it occurs: the array is **already sorted** (with the last element chosen as the pivot, as in 'Partition' above). Every partition is maximally unbalanced – one subproblem has size $n-1$ and the other has size Θ [C0011]:

$$T(n) = T(n-1) + \Theta(n)$$

Solving by iteration (this recurrence is not of the form $a \cdot T(n/b)$, so the Master Theorem does not apply):

$$T(n) = \Theta(n) + \Theta(n-1) + \Theta(n-2) + \dots + \Theta(1) = \Theta\left(\sum_{k=1}^n k\right) = \Theta(n^2)$$

Each level strips off only one element while doing $\Theta(n)$ partition work, giving **$\Theta(n^2)$** [C0011].

Comparison

Aspect	Best Case	Worst Case
Partition behaviour	Two nearly equal halves [C0010]	Completely unbalanced: sizes $(n-1, \Theta)$ [C0011]
Trigger	Balanced pivots	Array already sorted, last element as pivot [C0011]
Recurrence	$T(n) = 2T(n/2) + \Theta(n)$ [C0010]	$T(n) = T(n-1) + \Theta(n)$ [C0011]
Recursion depth	$\Theta(\log n)$ levels	$\Theta(n)$ levels
Complexity	$\Theta(n \log n)$	$\Theta(n^2)$

Key insight: the **same** algorithm ranges from $\Theta(n \log n)$ to $\Theta(n^2)$ purely because of pivot quality – the $\Theta(n)$ partition cost is constant across cases, but the recursion depth changes from logarithmic (balanced) to linear (unbalanced). This contrasts with merge sort, which is strictly $\Theta(n \log n)$ in all cases because it always splits the array into two halves [C0009].

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context fully covers Quick Sort's best/worst cases, the answer faithfully cites it plus the verified Master Theorem output, directly answers the question, and all recurrences and complexity bounds are correct.

Execution Latency: 29.42s

Q20: [GENERAL] — PYQ 2022-23 SECTION A 1(c)

Query Prompt: Discuss the skip list and its operations.

Retrieval Telemetry: Topic `tries_skip_lists` (Unit 2), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(2 on-topic), Max Similarity: 0.663

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0057	tries_skip_lists	2	0.03279	0.663	11.093	2.4 Tries and Skip Lists
C0056	tries_skip_lists	2	0.03128	0.4998	5.765	2.4 Tries and Skip Lists
C0018	binomial_fibonacci_heaps	2	0.02826	0.4335	5.491	2.3 Binomial and Fibonacci Heaps > Binomial Heap

Raw Generated Output:

```
# Skip List and its Operations

## 1. Definition and Structure

A skip list is a randomised alternative to a balanced tree [C0057]. Structurally, it is a stack of sorted linked lists:

- Level 0 holds every element of the list, in sorted order.
- Higher levels are progressively sparser: an element present at level i is promoted to level i+1 with probability p, usually p = 1/2 [C0057].

Because promotion is decided by probability (a coin flip) rather than by structural rules, the skip list achieves balance in expectation without any explicit rebalancing step. The expected number of levels is  $O(\log n)$  [C0057].

## 2. Search Operation

The search procedure given in the notes is [C0057]:

1. Start at the top level of the stack of lists.
2. Move right along the current level while the next key is smaller than the target.
3. When the next key is not smaller than the target, drop down one level and repeat.
4. The search terminates when the target is found (or the search position passes where it would be) at level 0.

Since each level is sorted, the "move right / drop down" rule skips over large portions of the list at high levels, giving the expected  $O(\log n)$  search cost [C0057].

## 3. Insert and Delete Operations

- Insert: the element is inserted into level 0, and then promoted upward level by level, each promotion succeeding with probability p (usually 1/2) [C0057]. This random promotion is what builds the higher levels.
- Delete: the notes give its expected cost as  $O(\log n)$  [C0057]. (The step-by-step deletion procedure – locating the node and unlinking it from each level it occupies – is not detailed in the provided context; only its expected complexity is stated.)

## 4. Complexity Summary

| Operation | Expected cost | Worst case |
|---|---|---|
| Search |  $O(\log n)$  |  $O(n)$  |
| Insert |  $O(\log n)$  |  $O(n)$  |
| Delete |  $O(\log n)$  |  $O(n)$  |
| Number of levels |  $O(\log n)$  | – |

All figures are as stated in [C0057]. Note the guarantee is expected (randomised)  $O(\log n)$ ; the worst case is  $O(n)$ , which occurs when the random promotions produce a degenerate, nearly single-level structure [C0057]. Because the worst case is  $O(n)$ , the  $O(\log n)$  bound holds only in expectation, not as a worst-case  $O(\log n)$  guarantee.

## 5. Practical Advantage

Skip lists need no rotations, which makes them simpler to implement than red-black trees [C0057]. Balance is maintained probabilistically at insertion time, so no rebalancing primitives (rotations, recolourings) are required at all.

## 6. What the Context Does Not Cover

The provided passage does not include: the space-complexity analysis of a skip list, pseudocode for insert/
```


delete, or the effect of choosing $p \neq 1/2$ on the level distribution. Only the structural description, the search rule, and the expected/worst-case complexities above are available in [C0057].

Metric Scores: context_relevance=66.7% | context_precision=100.0% | context_recall=70.0% | faithfulness=90.0% | answer_relevance=90.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer is accurate and well-grounded in C0057, but the context lacks detailed insert/delete procedures and space analysis, so recall is incomplete while the answer honestly flags those gaps.

Execution Latency: 14.69s

Q21: [GENERAL] — PYQ 2022-23 SECTION B 2(b)

Query Prompt: What are the differences between a Binomial Heap and a Fibonacci Heap?

Retrieval Telemetry: Topic binomial_fibonacci_heaps (Unit 2), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(3 on-topic), Max Similarity: 0.8138

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0018	binomial_fibonacci_heaps	2	0.03279	0.8138	22.686	2.3 Binomial and Fibonacci Heaps > Binomial Heap
C0019	binomial_fibonacci_heaps	2	0.03226	0.7709	18.472	2.3 Binomial and Fibonacci Heaps > Binomial Heap
C0017	binomial_fibonacci_heaps	2	0.03175	0.7371	15.481	2.3 Binomial and Fibonacci Heaps

Raw Generated Output:

```
# Binomial Heap vs. Fibonacci Heap

**Principle.** Both are advanced heap structures used to implement priority queues efficiently, especially for algorithms like Dijkstra's Shortest Path and Prim's MST, where the `Decrease-Key` operation is performed frequently [C0017]. The difference lies in how strictly the heap structure is maintained.

**Binomial Heap.** A binomial heap is implemented as a collection of binomial trees, where a binomial tree  $B_k$  is an ordered tree defined recursively. It supports union, insertion, and extracting the minimum in  $O(\log n)$  time; its merging operation is particularly fast compared to binary heaps [C0018].

**Fibonacci Heap.** A Fibonacci heap is a collection of rooted trees that are min-heap ordered. It is similar to a binomial heap but has a *looser* structure, allowing for delayed maintenance (lazy operations), which gives excellent amortized running times [C0018].

## Differences

| Aspect | Binomial Heap | Fibonacci Heap |
| --- | --- | --- |
| Structure | Collection of binomial trees ( $B_k$ , recursively defined ordered trees) | Collection of rooted min-heap ordered trees |
| Strictness | More rigidly maintained | Looser structure; delayed maintenance (lazy operations) |
| Insert |  $O(\log n)$  (union, insertion, extract-min all  $O(\log n)$ ) |  $O(1)$  amortized |
| Find-Min | — |  $O(1)$  amortized |
| Decrease-Key |  $O(\log n)$  (as with binary/binomial heaps) |  $O(1)$  amortized — the massive advantage |
| Extract-Min |  $O(\log n)$  |  $O(\log n)$  amortized |
| Running-time guarantee | Worst-case  $O(\log n)$  for union/insert/extract-min | Amortized bounds |

(All Fibonacci heap bounds are amortized; binomial heap bounds are as stated in [C0018].)

## Consequence

Because of the  $O(1)$  `Decrease-Key` operation, using a Fibonacci heap improves the time complexity of Dijkstra's algorithm from  $O((V + E) \log V)$  to  $O(V \log V + E)$  [C0019].

**Result:** The Fibonacci heap trades structural strictness for superior amortized costs —  $O(1)$  amortized Insert, Find-Min and Decrease-Key versus the binomial heap's  $O(\log n)$  operations — while both take  $O(\log n)$  for Extract-Min [C0018].
```

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=90.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer is fully grounded in the context and technically correct, with only a minor unsupported characterization of binomial heap bounds as worst-case guarantees.

Execution Latency: 12.23s

Q22: [GENERAL] — PYQ 2022-23 SECTION A 1(i)

Query Prompt: Define NP, NP-hard and NP-complete, and give an example of each.

Retrieval Telemetry: Topic `np_completeness` (Unit 5), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(3 on-topic), Max Similarity: 0.7352

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0041	np_completeness	5	0.03252	0.7352	19.969	5.2 NP-Completeness Theory > Complexity Class
C0042	np_completeness	5	0.03252	0.7218	24.474	5.2 NP-Completeness Theory > Complexity Class
C0040	np_completeness	5	0.03150	0.6795	13.527	5.2 NP-Completeness Theory

Raw Generated Output:

```
## Definitions and Examples

**1. NP (Nondeterministic Polynomial)** [C0041]

NP is the set of decision problems for which a *proposed* solution can be **verified** by a deterministic Turing machine in polynomial time. Note that P is a subset of NP [C0041].

*Example:* Since  $P \subseteq NP$  [C0041], any P problem such as **Shortest Path** is in NP. Also, every NP-Complete problem (e.g., SAT) is in NP by definition [C0042].

**2. NP-Hard** [C0041]

A problem X is NP-Hard if **every problem in NP can be reduced to X in polynomial time**. NP-hard problems do **not** have to be in NP themselves [C0041].

*Example:* **Traveling Salesperson Problem (TSP)** – it appears in the list of NP-Complete problems [C0042], and since every NP-Complete problem is NP-Hard by definition [C0042], TSP is NP-Hard. (The context does not give an example of a problem that is NP-Hard but *not* in NP; that information is missing.)

**3. NP-Complete (NPC)** [C0042]

A problem is NP-Complete if it is **both in NP AND NP-Hard**. These are the "hardest" problems in NP. If a polynomial time algorithm is found for any NPC problem, then  $P = NP$  [C0042].

*Example:* **SAT (Boolean Satisfiability Problem)** – the first problem proven to be NP-Complete, via the Cook-Levin Theorem [C0042]. Other examples given are TSP, 0/1 Knapsack (decision version), Vertex Cover, Clique, and Graph Coloring [C0042].

## Summary Table

| Class | Definition | Example |
| --- | --- | --- |
| NP | Solution verifiable in polynomial time by a deterministic TM | Shortest Path (as  $P \subseteq NP$ ) [C0041] |
| NP-Hard | Every problem in NP reduces to it in polynomial time; need not be in NP | TSP [C0041], [C0042] |
| NP-Complete | In NP **and** NP-Hard | SAT (Cook-Levin Theorem) [C0042] |

**Key relationship:** To prove a problem is NP-Complete, show it is in NP, then show a known NP-Complete problem transforms into it in polynomial time [C0042].
```

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context fully defines NP, NP-hard, and NP-complete with examples, and the answer faithfully, accurately, and completely presents them with grounded examples like SAT and TSP.

Execution Latency: 11.48s

Q23: [GENERAL] — PYQ 2022-23 SECTION A 1(g)

Query Prompt: Differentiate between Backtracking and Branch and Bound techniques.

Retrieval Telemetry: Topic `branch_and_bound` (Unit 4), Confidence: 0.7, Router: llm, Filter: priority:topic(3 on-topic), Max Similarity: 0.7069

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0055	branch_and_bound	4	0.03227	0.6315	12.43	4.7 Branch and Bound > Travelling Salesman Probl

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0054	branch_and_bound	4	0.03226	0.654	11.646	4.7 Branch and Bound
C0069	branch_and_bound	4	0.03126	0.5876	9.151	KCS-503 detailed syllabus

Raw Generated Output:

```
## Backtracking vs Branch and Bound

**Principle.** Both techniques search a state space tree – [C0054] introduces branch and bound with "Branch and bound **also** searches a state space tree," i.e., as the counterpart of the backtracking search already covered. The difference lies in *why* the tree is searched and *in what order* its nodes are explored.

| Aspect | Backtracking | Branch and Bound |
|---|---|---|
| **Purpose** | Feasibility-oriented: [C0054] defines branch and bound as the technique "used for optimisation **rather than feasibility**," drawing the contrast with backtracking, which answers whether a solution satisfying the constraints exists. | Optimisation-oriented: it searches for a minimum-cost (best) solution, not merely any feasible one [C0054]. |
| **Order of node exploration** | Not specified in the CONTEXT. | Explores nodes "in an order driven by a cost bound" [C0054]. Three strategies are given: **FIFO** (breadth first), **LIFO** (depth first), and **least cost**, in which the live node with the smallest bound becomes the next E-node [C0054]. |
| **Pruning / killing criterion** | Not specified in the CONTEXT. | Every live node carries a **lower bound** on the cost of any solution in its subtree; a node whose bound is no better than the best complete solution found so far is **killed** [C0054]. |
| **Bound computation** | No analogous machinery in the CONTEXT. | For TSP, the bound comes from **matrix reduction**: subtract row minima, then column minima; the total subtracted is the root bound. Branching on edge (i, j) sets row *i*, column *j* and entry (j, i) to infinity, re-reduces, and the child's bound = parent bound + edge cost + new reduction [C0055]. |
| **Typical example** | The syllabus lists Backtracking and Branch and Bound together with examples such as Travelling Salesman Problem, Graph Coloring, n-Queen Problem, Hamiltonian Cycles and Sum of Subsets [C0069]; the chunk does not individually assign these to one technique. | **Travelling Salesman Problem** is explicitly worked as a branch and bound problem [C0055], [C0069]. |
| **Worst-case behaviour** | Not given in the CONTEXT. | Pruning is effective in practice but "does not change the worst case": TSP remains NP-hard under branch and bound [C0055]. No Theta/O/Omega bound is stated in the CONTEXT, so the strongest claim available is that the worst case remains that of the underlying NP-hard problem. |

## What the CONTEXT does not contain

To make the comparison fully symmetric, the following are **missing** and I will not invent them:

1. A definition of backtracking's own mechanics – its node-expansion order (e.g., depth-first), its use of bounding/constraint functions, and its termination behaviour. Only the *implication* from "optimisation rather than feasibility" [C0054] and the word "also" in "also searches a state space tree" [C0054] characterise it indirectly.
2. Any asymptotic complexity expression (Theta/O/Omega) for backtracking; the only worst-case remark in the CONTEXT concerns branch and bound on TSP [C0055].

**Summary.** Both techniques explore a state space tree [C0054]; backtracking is the feasibility search, while branch and bound is its optimisation variant in which every live node carries a cost lower bound, nodes are expanded in bound-driven order (FIFO, LIFO or least cost), and any node whose bound cannot beat the best complete solution found so far is killed [C0054] – with bounds computed, for TSP, by matrix reduction [C0055].
```

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=40.0% | faithfulness=100.0% | answer_relevance=80.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer is faithful and technically correct, but the CONTEXT lacks backtracking's own mechanics (depth-first order, bounding functions), so the comparison is one-sided and several table cells are left unspecified.

Execution Latency: 26.44s

Q24: [GENERAL] — PYQ 2024-25 SECTION A 1(c)

Query Prompt: Derive the time complexity of Heap Sort.

Retrieval Telemetry: Topic `sorting_comparison` (Unit 1), Confidence: 0.95, Router: llm+lexical, Filter: priority:topic(6 on-topic), Max Similarity: 0.7553

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0012	sorting_comparison	1	0.03279	0.7553	11.44	1.3 Advanced Sorting Algorithms > Heap Sort
C0068	sorting_comparison	1	0.03105	0.6309	7.21	KCS-503 detailed syllabus

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0009	sorting_comparison	1	0.02939	0.6163	7.18	1.3 Advanced Sorting Algorithms > Merge Sort
C0010	sorting_comparison	1	0.02899	0.6276	5.332	1.3 Advanced Sorting Algorithms > Quick Sort
C0008	sorting_comparison	1	0.02786	0.6288	4.889	1.3 Advanced Sorting Algorithms

Raw Generated Output:

```
## Time Complexity of Heap Sort

**Principle.** Heap sort views the array as a nearly complete binary tree. It first builds a max-heap from the array, then repeatedly extracts the maximum element and restores the heap property [C0012].

**Algorithm** [C0012]:
```

Algorithm HeapSort(A) 1. Build-Max-Heap(A) 2. for i = A.length downto 2 3. exchange A[1] with A[i] 4. A.heap-size = A.heap-size - 1 5. Max-Heapify(A, 1)

```
**Derivation (cost of each phase):**

**Step 1 – Build-Max-Heap(A):** Building the heap takes  $O(n)$  time [C0012].

**Step 2 – Extraction loop (lines 2-5):** The loop runs for  $i = A.length$  downto  $2$ , i.e.  $n - 1$  iterations. Each iteration performs:
- one exchange  $A[1] \leftrightarrow A[i]$  –  $O(1)$ ,
- one call to  $\text{Max-Heapify}(A, 1)$  – since the array is a nearly complete binary tree, restoring the heap property takes time proportional to the height of the tree, i.e.  $O(\log n)$  per call [C0012].

**Step 3 – Total cost:**


$$T(n) = \underbrace{O(n)}_{\text{Build-Max-Heap}} + \underbrace{(n-1) \cdot O(\log n)}_{\text{extract + heapify}} = O(n \log n)$$


**Final Result:**

- Time complexity:  $O(n \log n)$  – building the heap takes  $O(n)$  time, and each of the  $n - 1$  calls to Max-Heapify takes  $O(\log n)$  time [C0012].
- Space complexity:  $O(1)$  – heap sort is an in-place algorithm [C0012].
```

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer faithfully derives $O(n \log n)$ from the fully provided heap sort context with correct algorithm, costs, and space complexity.

Execution Latency: 17.74s

Q25: [GENERAL] — PYQ 2024-25 SECTION A 1(g)

Query Prompt: Describe randomized algorithms and list a few examples.

Retrieval Telemetry: Topic randomized_algorithms (Unit 5), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(1 on-topic), Max Similarity: 0.7028

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0063	randomized_algorithms	5	0.03227	0.7028	9.035	5.4 Randomized Algorithms
C0041	np_completeness	5	0.02814	0.559	4.358	5.2 NP-Completeness Theory > Complexity Class
C0037	string_matching	5	0.02762	0.5614	3.48	5.1 String Matching Algorithms > Naive String Ma

Raw Generated Output:

```
## Randomized Algorithms

**Definition.** A randomized algorithm is an algorithm that makes random choices during its execution [C0063]. The random choices influence the algorithm's behaviour, so its running time (or, for some algorithms, its correctness) becomes a random variable rather than a fixed quantity.

**Classification.** Randomized algorithms fall into two classes [C0063]:
```

```
| Type | Correctness | Running time | Error handling |
|---|---|---|---|
| **Las Vegas** | Always correct | Random variable (varies run to run) | None needed – output is always right |
| **Monte Carlo** | May be wrong with small probability | Always finishes within a fixed bound | Repeating the algorithm lowers the error probability |
```

Examples. The CONTEXT names one concrete example:

- **Randomized Quicksort** – the variant of quicksort that picks a random pivot. It is a **Las Vegas** algorithm (always correct) with expected running time $O(n \log n)$ [C0063].

The CONTEXT describes Monte Carlo algorithms as a class but does not name any specific Monte Carlo example, so no further named examples can be listed from the given material.

Why randomization helps. Randomization removes the dependence on any particular bad input: because the algorithm's choices are random, no fixed input can force the bad case [C0063]. This is why randomized quicksort's expected time $O(n \log n)$ holds regardless of the input ordering, unlike a deterministic pivot rule whose worst case can be triggered by a specific input.

Metric Scores: context_relevance=33.3% | context_precision=100.0% | context_recall=70.0% | faithfulness=100.0% | answer_relevance=70.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context fully supports the definition, Las Vegas/Monte Carlo classification, and randomized quicksort example, and the answer is faithful and accurate, but only one concrete example is available so the 'list a few examples' request is only partially met.

Execution Latency: 5.74s

Q26: [GENERAL] — authored

Query Prompt: State the properties that every Red-Black tree must satisfy.

Retrieval Telemetry: Topic red_black_trees (Unit 2), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(5 on-topic), Max Similarity: 0.7032

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0014	red_black_trees	2	0.03279	0.7032	18.551	2.1 Red-Black Trees > Properties of Red-Black Tr
C0013	red_black_trees	2	0.03226	0.6703	13.003	2.1 Red-Black Trees
C0060	red_black_trees	2	0.02964	0.4472	7.571	2.5 AVL Trees and Rotations
C0058	red_black_trees	2	0.02904	0.4697	4.429	2.5 AVL Trees and Rotations
C0059	red_black_trees	2	0.02721	0.4544	2.898	2.5 AVL Trees and Rotations

Raw Generated Output:

Definition. A Red-Black tree is a binary search tree with one extra bit of storage per node – its color, which is either RED or BLACK. By constraining the node colors on any simple path from the root to a leaf, the tree is kept approximately balanced: no path is more than twice as long as any other, so insertion, deletion and searching take $O(\log n)$ time in the worst case. [C0013]

Properties. Every Red-Black tree must satisfy the following five properties: [C0014]

1. Every node is either red or black.
2. The root is black.
3. Every leaf (NIL pointer) is black.
4. If a node is red, then both its children are black – i.e., no two adjacent red nodes on a path.
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes (the black-height).

Restoring the properties. When nodes are inserted or deleted, these properties may be violated. They are restored using two operations – recoloring and rotations (Left-Rotate and Right-Rotate). Rotations alter the structure of the tree while preserving the binary search tree property. [C0014]

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% | faithfulness=100.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer lists all five red-black properties exactly as given in C0014, with correct supporting definition from C0013 and no unsupported claims.

Execution Latency: 6.35s

Q27: [GENERAL] — PYQ 2022-23 SECTION B 2(e)

Query Prompt: Explain the Rabin-Karp string matching algorithm and what a spurious hit is.

Retrieval Telemetry: Topic `string_matching` (Unit 5), Confidence: 0.99, Router: llm+lexical, Filter: priority:topic(4 on-topic), Max Similarity: 0.7439

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0038	string_matching	5	0.03279	0.7439	25.563	5.1 String Matching Algorithms > Rabin-Karp Algo
C0039	string_matching	5	0.03175	0.6232	8.522	5.1 String Matching Algorithms > Knuth-Morris-Pr
C0037	string_matching	5	0.03175	0.5632	13.063	5.1 String Matching Algorithms > Naive String Ma
C0036	string_matching	5	0.03175	0.5889	10.073	5.1 String Matching Algorithms

Raw Generated Output:

```
## Rabin-Karp String Matching Algorithm

**Problem setting.** String matching deals with finding all occurrences of a shorter pattern string P (length m) within a longer text string T (length n) [C0036].

**Core idea.** Rabin-Karp improves string matching by using a hash function. Instead of comparing character by character initially, it compares the hash value of the pattern with the hash value of the current substring (window) of the text [C0038].

**Working.**
1. Compute the hash value of the pattern P (length m).
2. Compute the hash value of the first m-character substring of T.
3. Slide the window across T one position at a time. Using a **rolling hash mechanism**, the hash of the next substring is computed in O(1) time from the previous one (drop the leftmost character, add the new rightmost character) [C0038].
4. Whenever the hash of the window equals the hash of the pattern, perform a character-by-character comparison to confirm a true match [C0038].

**Spurious hit.** A spurious hit is a situation where the hash value of the text window matches the hash value of the pattern, but the actual characters do not match – i.e., two different strings collide on the same hash value. This is exactly why the algorithm does a character-by-character comparison after every hash match: to confirm the match and avoid spurious hits [C0038].

**Complexity.**
- Average case:  $O(n + m)$  [C0038].
- Worst case:  $O(nm)$  – e.g., when many spurious hits force repeated explicit comparisons [C0038].

**Comparison with naive method.** The naive approach slides the pattern over the text one character at a time and checks for a match, taking  $O((n - m + 1) \cdot m)$  time [C0037]. Rabin-Karp improves on this in the average case by replacing most character comparisons with O(1) rolling-hash comparisons, though its worst case remains  $O(nm)$  [C0038].
```

Metric Scores: context_relevance=100.0% | context_precision=100.0% | context_recall=90.0% | faithfulness=95.0% | answer_relevance=100.0% | semantic_accuracy=100.0%

Judge Audit Reason: The answer fully explains Rabin-Karp and spurious hits using the provided context, with only minor details like the rolling hash update rule and spurious-hit example being standard extensions of the context.

Execution Latency: 17.38s

Q28: [DIAGRAM] — PYQ 2022-23 SECTION B 2(c)

Query Prompt: Illustrate the stages of Kruskal's algorithm on a weighted graph, showing which edge is added at each step.

Retrieval Telemetry: Topic `mst` (Unit 3), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(1 on-topic), Max Similarity: 0.623

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0033	mst	3	0.03279	0.623	22.183	4.4 Graph Algorithms: Traversals & MST > Minimum
C0026	huffman_coding	3	0.02881	0.543	10.445	3.4 Huffman Coding
C0034	dijkstra	3	0.02879	0.5477	10.415	4.5 Single Source Shortest Path > Dijkstra's Alg

Diagrams Offered: DIAG_DAA_U3_MST_KRUSKAL_01 | **Emitted:** DIAG_DAA_U3_MST_KRUSKAL_01

Raw Generated Output:

```
## Kruskal's Algorithm – Stages of MST Construction

**Principle.** A Minimum Spanning Tree of a connected, undirected, weighted graph is a subgraph that is a tree, includes all the vertices, and has the minimum possible total edge weight [C0033]. Kruskal's algorithm builds it greedily, edge by edge.

**Stages of the algorithm** [C0033]:

1. **Sort:** Sort all edges of the graph in non-decreasing order of their weight.
2. **Select:** Pick the smallest edge from the sorted list.
3. **Cycle test:** Check whether this edge forms a cycle with the edges already included in the MST, using a **Disjoint Set Union (DSU)** data structure.
4. **Add or reject:** If it does not form a cycle, add it to the MST; otherwise discard it.
5. **Repeat** steps 2–4, continuing through the sorted edge list, until the tree spans all the vertices (i.e., it is a tree that includes all the vertices, as required by the MST definition [C0033]).

**Illustration of the stages.** The successive stages – the sorted edge order being scanned and the lightest acyclic edges being added one at a time into the growing forest, which merges into a single spanning tree – are shown in [Diagram: DIAG_DAA_U3_MST_KRUSKAL_01]. Each stage of that diagram corresponds to one iteration of the loop above: the current lightest edge is tested against the DSU, and because it is acyclic it is added, so the number of components in the forest decreases by one until only the MST remains.

**What is missing for a numeric trace.** The CONTEXT does not supply a concrete graph instance – no vertex set and no edge list with weights. Without that edge list, the sorted order and the specific edges added at each step (e.g., "add edge (u, v) of weight w") cannot be named; only the generic stage-by-stage construction shown in the diagram can be described. To produce a fully worked numeric trace, the missing input needed is: the weighted graph's edge list (u, v, weight) for each edge.

**Complexity.** The dominant cost is sorting the edges, giving  $O(E \log E)$  [C0033].
```

Metric Scores: context_relevance=33.3% | context_precision=100.0% | context_recall=40.0% | faithfulness=90.0% | answer_relevance=50.0% | semantic_accuracy=100.0%

Judge Audit Reason: The context explains Kruskal's algorithm but provides no concrete weighted graph, so the required step-by-step edge additions cannot be traced; the answer is faithful and correct but only generically addresses the illustration request.

Execution Latency: 13.05s

Q29: [DIAGRAM] — PYQ 2022-23 SECTION A 1(h)

Query Prompt: Draw the state space tree explored by backtracking for the 4-Queens problem and explain the bounding function.

Retrieval Telemetry: Topic backtracking (Unit 4), Confidence: 0.98, Router: llm+lexical, Filter: priority:topic(5 on-topic), Max Similarity: 0.713

Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0049	backtracking	4	0.03252	0.713	24.185	4.6 Backtracking
C0050	backtracking	4	0.03252	0.6764	26.376	4.6 Backtracking > n-Queens
C0051	backtracking	4	0.03037	0.5595	16.889	4.6 Backtracking > Graph Colouring
C0053	backtracking	4	0.02963	0.5604	10.889	4.6 Backtracking > Hamiltonian Cycles
C0052	backtracking	4	0.02879	0.5434	10.088	4.6 Backtracking > Sum of Subsets

Diagrams Offered: DIAG_DAA_U4_NQUEEN_TREE_01 | **Emitted:** DIAG_DAA_U4_NQUEEN_TREE_01

Raw Generated Output:

```
## 1. The problem and the method

**4-Queens:** place 4 queens on a 4 × 4 board so that no two share a row, column or diagonal. Since queens are placed one per row, a solution is a vector (x1, x2, x3, x4) where xk is the column of the queen in row k [C0050].

**Backtracking** performs a depth-first traversal of the implicit state space tree: a partial solution is extended one component at a time, and a bounding function decides whether the partial solution can still lead to a feasible complete solution; if not, the whole subtree is pruned and the search backtracks [C0049].

## 2. The bounding function
```

For n-Queens the bounding function is the conflict test `Place(k, j)` for putting queen k in column j. Queen k in column j **conflicts** with an earlier queen i sitting in column x[i] exactly when [C0050]:

```
- **Same column:** x[i] = j, or
- **Same diagonal:** |x[i] - j| = |i - k| (the column distance equals the row distance).
```

If any earlier queen conflicts, the partial vector ($x_1, \dots, x_k$) cannot be extended to a complete non-attacking placement, so the node is killed and the entire subtree below it is pruned [C0049, C0050]. Because the constraints only relate pairs of already-placed queens, this local test is exact: a pruned node can never contain a solution.

3. State space tree for n = 4

The full tree for the explored search is shown in [Diagram: DIAG_DAA_U4_NQUEEN_TREE_01]. The same exploration, with every bounding-function rejection marked X, is:

```

root (∅) |— x1 = 1 | |— x2 = 1 X same column as x1 | |— x2 = 2 X diagonal with x1 (|1-2| = |1-2|) | |— x2 = 3
— row 3: x3=1 X col(x1), x3=2 X diag(x2), x3=3 X col(x2), | | x3=4 X diag(x2) → dead end, backtrack | |— x2 = 4 | |—
x3 = 2 — row 4: x4=1 X col(x1), x4=2 X col(x3), x4=3 X diag(x3), | x4=4 X col(x2) → dead end, backtrack to root |— x1 = 2
| |— x2 = 1 X diagonal with x1 | |— x2 = 2 X same column as x1 | |— x2 = 3 X diagonal with x1 | |— x2 = 4 | |—
x3 = 1 |

```

```

**Metric Scores:** context_relevance=100.0% | context_precision=100.0% | context_recall=100.0% |
faithfulness=100.0% | answer_relevance=60.0% | semantic_accuracy=70.0%
**Judge Audit Reason:** The answer is fully grounded in the context and the shown tree nodes are correct,
but it is truncated mid-tree, never completing the state space tree or exhibiting the two solutions
(2,4,1,3) and (3,1,4,2), so the question is only partially answered.
**Execution Latency:** 86.94s

```

Q30: [DIAGRAM] – authored

```

**Query Prompt:** Show the LCS table with direction arrows for two strings and explain how the arrows
recover the subsequence.

```

```

**Retrieval Telemetry:** Topic `lcs` (Unit 4), Confidence: 0.98, Router: llm+lexical, Filter:
priority:topic(2 on-topic), Max Similarity: 0.7724

```

Retrieved Context Chunks:

Chunk ID	Topic	Unit	RRF Score	Dense	BM25	Section Heading
C0045	lcs	4	0.03252	0.7724	27.08	4.2b Longest Common Subsequence
C0046	lcs	4	0.03252	0.7123	33.341	4.2b Longest Common Subsequence
C0050	backtracking	4	0.02652	0.5184	7.943	4.6 Backtracking > n-Queens

Solvers Executed: `lcs`

Diagrams Offered: DIAG_DAA_U4_LCS_TABLE_01 | **Emitted:** DIAG_DAA_U4_LCS_TABLE_01

Raw Generated Output:

```
```markdown
```

# Longest Common Subsequence – Table with Direction Arrows

**Problem instance:** X = ABCBDAB (m = 7), Y = BDCABA (n = 6).

## 1. Principle

Let  $c[i][j]$  be the length of an LCS of the prefixes  $X_i$  and  $Y_j$ . The recurrence is [C0045]:

```

- $c[i][j] = 0$ if $i = 0$ or $j = 0$
- $c[i][j] = c[i-1][j-1] + 1$ if $i, j > 0$ and $x_i = y_j$
- $c[i][j] = \max(c[i-1][j], c[i][j-1])$ if $i, j > 0$ and $x_i \neq y_j$

```

A parallel table  $b[i][j]$  stores a direction arrow [C0045]:

```

- ↖ diagonal** when $x_i = y_j$ (the character belongs to the LCS)
- ↑ up** when $c[i-1][j] \geq c[i][j-1]$
- ← left** otherwise

```

## 2. Completed table (values with arrows)



The table below corresponds to the standard LCS table with direction arrows, where the highlighted cells form the traceback path [Diagram: DIAG\_DAA\_U4\_LCS\_TABLE\_01]. Bold cells are the cells visited during recovery.

```
| c[i][j] | j=0 | j=1 (B) | j=2 (D) | j=3 (C) | j=4 (A) | j=5 (B) | j=6 (A) |
|---|---|---|---|---|---|---|
| **i=0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **i=1 (A)** | 0 | 0 ↑ | 0 ↑ | 0 ↑ | *1 ↖** | 1 ← | 1 ↖ |
| **i=2 (B)** | 0 | *1 ↖** | *1 ←** | 1 ← | 1 ↑ | 2 ↖ | 2 ← |
| **i=3 (C)** | 0 | 1 ↑ | 1 ↑ | *2 ↖** | *2 ←** | 2 ↑ | 2 ↑ |
| **i=4 (B)** | 0 | 1 ↖ | 1 ↑ | 2 ↑ | 2 ↑ | *3 ↖** | 3 ← |
| **i=5 (D)** | 0 | 1 ↑ | 2 ↖ | 2 ↑ | *2 ↑** | 3 ↑ | 3 ↑ |
| **i=6 (A)** | 0 | 1 ↑ | 2 ↑ | 2 ↑ | 3 ↖ | *3 ↑** | *4 ↖** |
| **i=7 (B)** | 0 | 1 ↖ | 2 ↑ | 2 ↑ | 3 ↑ | 4 ↖ | *4 ↑** |
```

The entry in the bottom-right corner is **`c[7][6] = 4`**, the length of the LCS [C0046].

### ## 3. How the arrows recover the subsequence

Starting at `b[m][n] = b[7][6]`, follow the arrows backwards, emitting a character on every diagonal arrow; the characters come out in reverse order [C0046]:

```
| Step | Cell (i, j) | Arrow | Action |
|---|---|---|---|
| 1 | (7, 6) | ↑ | move up to (6, 6) |
| 2 | (6, 6) | ↖ | emit x6 = **A**, move to (5, 5) |
| 3 | (5, 5) | ↑ | move up to (4, 5) |
| 4 | (4, 5) | ↖ | emit x4 = **B**, move to (3, 4) |
| 5 | (3, 4) | ← | move left to (3, 3) |
| 6 | (3, 3) | ↖ | emit x3 = **C**, move to (2, 2) |
| 7 | (2, 2) | ← | move left to (2, 1) |
| 8 | (2, 1) | ↖ | emit x2 = **B**, move to (1, 0); i = 0 → stop |
```

Characters emitted along the walk: A, B, C, B – read in reverse: **`**BCBA**`**.

### ## 4. Result

- **`**LCS length:** c[7][6] = 4`**  
 - **`**One LCS:** **BCBA**`** (BDAB and BCAB are also longest common subsequences of length 4) [C0046]

**`**Complexity:**`** each of the `m.n` entries is computed in constant time, so the time complexity is **`**O(mn)**`**; space is **`**O(mn)**`**, reducible to **`**O(min(m, n))**`** if only the length is required [C0046].

**Metric Scores:** context\_relevance=66.7% | context\_precision=100.0% | context\_recall=100.0% | faithfulness=95.0% | answer\_relevance=100.0% | semantic\_accuracy=100.0%

**Judge Audit Reason:** The table values, arrows, traceback, LCS BCBA, and complexity all match the verified solver output and context exactly; the only blemish is a fabricated diagram reference tag.

**Execution Latency:** 31.71s

### Q31: [OOS] — authored

**Query Prompt:** How do I bake sourdough bread at home?

**Retrieval Telemetry:** Topic None (Unit None), Confidence: 0.0, Router: llm:none, Filter: none, Max Similarity: 0.4689

#### Retrieved Context Chunks:

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0067	asymptotic_notation	1	0.02991	0.3963	3.863	KCS-503 detailed syllabus
C0030	knapsack_01	4	0.02819	0.4108	2.35	4.3 0/1 Knapsack Problem
C0047	floyd_warshall	4	0.02721	0.4028	2.333	4.4b All Pairs Shortest Paths: Floyd-Warshall
C0045	lcs	4	0.02659	0.383	2.375	4.2b Longest Common Subsequence
C0010	sorting_comparison	1	0.02632	0.3906	2.034	1.3 Advanced Sorting Algorithms > Quick Sort

#### Raw Generated Output:

This question falls outside the AKTU Design and Analysis of Algorithms (KCS-503) syllabus material indexed by this system, so there is no retrieved context to ground an answer in. Query did not map to any syllabus topic and best similarity 0.47 is below 0.58. Please ask about a DAA topic such as asymptotic notation, recurrences, sorting, advanced data structures, divide and conquer, greedy methods, dynamic programming, backtracking, string matching or NP-completeness.

**Metric Scores:****Judge Audit Reason:** out of scope, graded on refusal only**Execution Latency:** 1.5s**Q32: [OOS] — authored****Query Prompt:** Explain the self-attention mechanism used in transformer neural networks.**Retrieval Telemetry:** Topic `None` (Unit None), Confidence: 0.0, Router: llm:none, Filter: none, Max Similarity: 0.484**Retrieved Context Chunks:**

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0021	greedy_general	3	0.03175	0.4728	4.638	3.2 Greedy Algorithms Overview
C0056	tries_skip_lists	2	0.02944	0.4604	3.128	2.4 Tries and Skip Lists
C0025	huffman_coding	3	0.02941	0.4583	3.178	3.4 Huffman Coding
C0054	branch_and_bound	4	0.02921	0.4452	5.079	4.7 Branch and Bound
C0017	binomial_fibonacci_heaps	2	0.02814	0.4416	3.884	2.3 Binomial and Fibonacci Heaps

**Raw Generated Output:**

This question falls outside the AKTU Design and Analysis of Algorithms (KCS-503) syllabus material indexed by this system, so there is no retrieved context to ground an answer in. Query did not map to any syllabus topic and best similarity 0.48 is below 0.58. Please ask about a DAA topic such as asymptotic notation, recurrences, sorting, advanced data structures, divide and conquer, greedy methods, dynamic programming, backtracking, string matching or NP-completeness.

**Metric Scores:****Judge Audit Reason:** out of scope, graded on refusal only**Execution Latency:** 0.66s**Q33: [OOS] — authored****Query Prompt:** Who is the current Vice Chancellor of AKTU and what is the exam fee for the fifth semester?**Retrieval Telemetry:** Topic `None` (Unit None), Confidence: 0.0, Router: llm:none, Filter: none, Max Similarity: 0.3654**Retrieved Context Chunks:**

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0006	recurrences	1	0.03125	0.3554	11.588	1.2 Recurrences & Master Theorem > The Master Th
C0005	recurrences	1	0.03058	0.3364	11.677	1.2 Recurrences & Master Theorem > The Master Th
C0038	string_matching	5	0.02863	0.3048	11.999	5.1 String Matching Algorithms > Rabin-Karp Algo
C0047	floyd_warshall	4	0.02686	0.3246	10.432	4.4b All Pairs Shortest Paths: Floyd-Warshall
C0065	job_sequencing	3	0.02565	0.3091	10.416	3.3b Job Sequencing with Deadlines

**Raw Generated Output:**

This question falls outside the AKTU Design and Analysis of Algorithms (KCS-503) syllabus material indexed by this system, so there is no retrieved context to ground an answer in. Best passage similarity 0.37 below floor 0.45. Please ask about a DAA topic such as asymptotic notation, recurrences, sorting, advanced data structures, divide and conquer, greedy methods, dynamic programming, backtracking, string matching or NP-completeness.

**Metric Scores:****Judge Audit Reason:** out of scope, graded on refusal only**Execution Latency:** 0.78s**Q34: [OOS] — authored****Query Prompt:** Describe the CAP theorem and how distributed databases handle partition tolerance.**Retrieval Telemetry:** Topic `None` (Unit None), Confidence: 0.0, Router: llm:none, Filter: none, Max Similarity: 0.5502

**Retrieved Context Chunks:**

CHUNK ID	TOPIC	UNIT	RRF SCORE	DENSE	BM25	SECTION HEADING
C0006	recurrences	1	0.03178	0.514	10.53	1.2 Recurrences & Master Theorem > The Master Th
C0005	recurrences	1	0.03128	0.51	9.974	1.2 Recurrences & Master Theorem > The Master Th
C0007	recurrences	1	0.02914	0.4896	6.32	1.2 Recurrences & Master Theorem > Example of Ma
C0028	dp_principles	4	0.02853	0.5224	3.127	4.1 Dynamic Programming Principles
C0062	divide_and_conquer	3	0.02808	0.5032	3.196	3.1b Divide and Conquer: Strassen and Convex Hul

**Raw Generated Output:**

This question falls outside the AKTU Design and Analysis of Algorithms (KCS-503) syllabus material indexed by this system, so there is no retrieved context to ground an answer in. Query did not map to any syllabus topic and best similarity 0.55 is below 0.58. Please ask about a DAA topic such as asymptotic notation, recurrences, sorting, advanced data structures, divide and conquer, greedy methods, dynamic programming, backtracking, string matching or NP-completeness.

**Metric Scores:**

**Judge Audit Reason:** out of scope, graded on refusal only

**Execution Latency:** 0.62s

**VI. QUANTITATIVE EVALUATION SCORECARD**

The system was benchmarked against 34 ground-truth test queries over 383.6s wall-clock time using generation model `z-ai/glm-5.3-flash` via `openai_compat`. Metrics are categorized into Retrieval, Generation, and Domain-Specific stages according to the standardized calibration brief.

**A. Master Evaluation Scorecard**

STAGE	METRIC NAME	MEASURED SCORE	PASS CRITERION	AUDIT RESULT	VERIFICATION OBJECTIVE
Retrieval	Context Relevance	71.1%	>= 60%	PASS	High relevance; retrieved chunks pertinent to the query
Retrieval	Context Recall	91.3%	>= 80%	PASS	Context contains the algorithmic steps needed for a complete answer
Retrieval	Context Precision & Ranking	100.0%	>= 90%	PASS	Top-1 / Top-2 chunks carry the primary solution substance
Generation	Faithfulness / Groundedness	95.8%	>= 80%	PASS	Zero hallucinations; every claim grounded in retrieved context
Generation	Answer Relevance	95.0%	>= 85%	PASS	Direct, exhaustive answer addressing all question parts
Generation	Semantic & Mathematical Accuracy	99.0%	>= 90%	PASS	Mathematically sound; exact complexity notation
Specialized	Cross-Topic Disambiguation Rate	100.0%	>= 90%	PASS	Intended topic resolved without cross-unit collision
Specialized	Diagram ID Linkage Rate	100.0%	>= 90%	PASS	Correct Diagram ID emitted for visual queries
Specialized	Negative / Out-of-Scope Robustness	100.0%	>= 90%	PASS	Explicit rejection of out-of-scope queries, no hallucinated answer

**Summary: 9 of 9 standardized checklist metrics passed.**

**B. Granular Breakdown by Query Category**

CATEGORY	CTX RELEVANCE	CTX PRECISION	FAITHFULNESS	ANSWER RELEVANCE	SEMANTIC ACC.	ROBUSTNESS
DIAGRAM	80.0%	100.0%	97.0%	82.0%	94.0%	100.0%
DISAMBIGUATION	50.0%	100.0%	90.0%	100.0%	100.0%	100.0%
GENERAL	88.9%	100.0%	97.2%	93.3%	100.0%	100.0%
NUMERIC	66.7%	100.0%	99.4%	100.0%	100.0%	100.0%
OOS	n/a	n/a	n/a	n/a	n/a	100.0%

*Supporting Criterion: Mandatory keyword coverage in generated answers: 100.0%.*

## VII. FAILURE MODE ANALYSIS AND LIMITATIONS

### A. Observed Defect Audit

QUERY ID	CATEGORY	FAILING STAGE	OBSERVED ROOT CAUSE
Q12	DISAMBIGUATION	Generation	faithfulness 40.0%: The context gives the pseudocode, the $ V -1$ -edge shortest-path justification, and the verified trace, but never defines the relaxation test/
Q12	DISAMBIGUATION	Retrieval	context_recall 60.0%: The context gives the pseudocode, the $ V -1$ -edge shortest-path justification, and the verified trace, but never defines the relaxation test/
Q23	GENERAL	Retrieval	context_recall 40.0%: The answer is faithful and technically correct, but the CONTEXT lacks backtracking's own mechanics (depth-first order, bounding functions),
Q28	DIAGRAM	Retrieval	context_recall 40.0%: The context explains Kruskal's algorithm but provides no concrete weighted graph, so the required step-by-step edge additions cannot be trac

### B. Identified Systemic Limitations

1. **Corpus Depth and Sparsity:** Baseline student notes provide minimal passages for several topics (e.g., Bellman-Ford and Prim's MST). Retrieval falls back to unit priors when fewer than 3 chunks match, placing an upper ceiling on pure passage relevance.
2. **Authored Supplement Dependency:** Ten syllabus topics absent in the baseline notes were authored in `notes_supplement.md`. While rigorously tagged, answers on these topics cite the supplement rather than student notes.
3. **Reasoning Token Budgeting:** On reasoning-heavy LLM endpoints, token limits cover reasoning tokens; structured-output calls (routing and judging) require low reasoning effort to prevent token exhaustion.
4. **Out-of-Scope Similarity Thresholding:** Off-syllabus queries lexically adjacent to computer science concepts require strict similarity gating calibrated against empirical negative test queries.

### C. Proposed Architectural Mitigations

- **Corpus Expansion:** Indexing full textbook chapters (CLRS 4th ed.) to elevate per-topic chunk volume.
- **Cross-Encoder Reranking:** Introducing a second-stage cross-encoder over the fused top-20 candidates for enhanced precision.
- **Runtime Grounding Enforcement:** Integrating `RubricMiddleware` to dynamically evaluate and re-prompt ungrounded generations during inference.